

MASTER'S THESIS 2025

Real-Time Geometry Reconstruction with Compute Shader Tessellation

Jintao Yu

Elektroteknik
Datateknik

ISSN XXXX-XXXX
LU-CS-EX: XXXX-XX

DEPARTMENT OF DESIGN SCIENCE
LTH | LUND UNIVERSITY



MASTER'S THESIS

LU-CS-EX: XXXX-XX

**Real-Time Geometry Reconstruction
with Compute Shader Tessellation**

Jintao Yu

Real-Time Geometry Reconstruction with Compute Shader Tessellation

Master's Thesis

Jintao Yu

jintaoyuemail@gmail.com

May 27, 2025

Master's thesis work carried out at Electronic Arts.

Supervisors: Kindahl Christian, ckindahl@ea.com

Michael Doggett, michael.doggett@cs.lth.se

Examiner: Mattias Wallergård, mattias.wallergard@design.lth.se

Abstract

Traditional hardware-based tessellation has long been used for surface refinement in real-time rendering, yet suffers from limited flexibility, lack of programmability, and inefficient handling of geometry reuse. These constraints become more pronounced when adapting to modern rendering demands such as dynamic displacement, flexible LOD strategies, and memory-aware mesh reconstruction. To overcome these limitations, we introduce a compute-shader-based geometry reconstruction framework that leverages precomputed tessellation patterns to dynamically refine coarse meshes in real time.

The proposed system is fully programmable, supports arbitrary topologies, and enables GPU-side generation of high-density primitives with efficient memory usage. Through experiments, we compare our method against widely adopted approaches. We demonstrate that our method reduces mesh loading times and improves rendering efficiency while maintaining comparable visual quality. This work contributes a practical, scalable tessellation solution, and offers a step toward flexible, shader-driven geometry pipelines suited for future real-time graphics applications.

Keywords: Computer Graphics, Geometry Processing, GPU-based Tessellation, Mesh Refinement

Acknowledgements

Contents

1	Introduction	9
1.1	Background	9
1.2	Aim	10
1.3	Research Questions	10
1.4	Contribution	10
1.5	Sustainable Development Goals	11
1.6	Ethics	12
2	Theory and Related Work	13
2.1	Tessellation	13
2.2	Geometry representation	14
2.2.1	Topology	15
2.2.2	Polygon	15
2.3	Compute shader	16
2.3.1	Pipeline and Architecture	16
2.3.2	Data Access and Manipulation	17
2.4	Displacement mapping	18
2.5	Related work	19
2.5.1	Prior Refinement Schemes	19
2.5.2	Hardware Tessellation	20
2.5.3	Nanite	21
2.5.4	Micro-Mesh and Mega Geometry	21
3	Methodology	23
3.1	Literature Study	23
3.2	Implementation Overview	23
3.3	Tools and Framework	24

4	Implementation	25
4.1	Pipeline Overview	25
4.1.1	Resources Overview	25
4.1.2	Data-preprocessing	26
4.1.3	Real time framework	27
4.2	Patterns Generation	28
4.3	Visibility and tess rate determination	31
4.4	Tessellation and Displacement	32
4.4.1	Vertices Deduplication	34
4.5	Normal Re-Calculation	34
5	Evaluation and Results	37
5.1	Visual Quality Analysis	37
5.1.1	Visibility and Tessellation	37
5.1.2	Displacement mapping	40
5.1.3	Normal Recalculation	41
5.2	Performance Analysis	43
6	Limitation and Future work	47
6.1	Data Optimization	47
6.2	Adaptive Patterns	48
6.3	Generic Vertices Deduplication	48
6.4	Next-Gen Geometry Pipeline	50
6.5	Visual effect	51
7	Conclusion	55
7.1	Key Findings	55
7.2	Research Questions	55
	References	57

List of abbreviations

Chapter 1

Introduction

This chapter briefly introduces the background of geometry refinement techniques, discusses the main challenges in the current field, and outlines the objectives of this thesis.

1.1 Background

Reproducing high fidelity geometry in real time play a vital role in computer graphics. In recent years, the demand for real-time photorealistic rendering has driven a surge of interest in high-resolution geometry synthesis, particularly in the context of entertainment industry, AR/VR and virtual production pipelines.

While providing benefits such as better visual effects, high-fidelity geometry also brings several challenges due to its massive data size. These challenges include reduced editability due to complex mesh topologies that are difficult to animate or manipulate in real-time engines. Increased computational burden for examples more expensive cost of lighting calculations, ray tracing, accurate collision detection, and animation of dense meshes, as well as greater memory requirements for vertices data and textures. Higher bandwidth is required between CPU and GPU and IO overhead among different shader stages and GPU memory [1].

Generating additional triangles in real-time on the gpu, so-call tessellation, is gradually becoming one of the solution for rendering a large number of triangles in real-time. In the development of geometry processing technology in these decades, this concept has given rise to a wealth of research results and technical solutions [2–7], especially Hardware Tessellation [8].

However, in terms of flexibility and utilization of gpu resources, Hardware Tessellation in the traditional gpu rendering pipeline is no longer suitable for the iteration and development of modern gpu architectures. With the introduction of the compute shader [9]

and the concept of GPU driven pipeline [10], it has become possible to use gpu resources flexibly and efficiently. Based on this trend, this paper prototype a customized compute shaders pipeline to control the segmentation behavior and detail sampling process, with the intention of exploring and implementing a rendering pipeline which utilizes the compute shader tessellation with displacement mapping to restore high-precision models in real time.

1.2 Aim

This thesis explores the latest real-time surface tessellation techniques, with a specific focus on compute shader implementations. Building upon this foundation, it further develops and prototypes a rendering pipeline aimed at supporting high-fidelity geometries. Additionally, the advantages and disadvantages of compute shader-based subdivision techniques are evaluated, including their impact on rendering performance, frame rates and etc. compare to the traditional hardware tessellation and even more advance technique Nanite [11].

1.3 Research Questions

Based on the objectives and challenges outlined above, this thesis aims to address the following research questions:

1. How can compute shaders be effectively utilized to implement an efficient and flexible real-time surface tessellation method?
2. What are the trade-offs between computational complexity and rendering performance when employing a compute shader-based tessellation solution compared to other solutions?
3. Is it feasible to achieve high-fidelity geometry reconstruction in real-time rendering using software-based tessellation combined with displacement mapping? What are the technical challenges involved in this approach?

1.4 Contribution

This thesis presents a real-time geometry reconstruction pipeline based on compute shader-driven tessellation, enabling dynamic surface refinement and displacement mapping directly on the GPU. It introduces a reusable, pattern-based tessellation method implemented entirely in compute shaders, capable of generating high-density geometry with arbitrary input topology.

The proposed pipeline is designed for practical use, with particular attention to memory efficiency and visual fidelity. Finally, the system is evaluated through detailed comparisons with traditional approaches such as hardware tessellation and Nanite, demonstrating its effectiveness in both performance and visual quality.

1.5 Sustainable Development Goals

The Sustainable Development Goals, short for SDGs, are 17 global development goals put forward by the United Nations in 2015 [12], aiming to eradicate poverty, protect the planet, and promote peace and prosperity for all humanity by 2030. Our project meet the targets of SDG 4: Quality Education and SDG 9: Industry, Innovation and Infrastructure.

Quality Education



Figure 1: Quality Education

My project has been able to significantly enhance the immersive experience of education and training systems through in-depth research and promotion of real-time high-fidelity graphic rendering technologies, especially in VR/AR teaching platforms. In areas such as medical training, engineering simulation, and historical re-enactment, high-quality graphics rendering combined with virtual reality and augmented reality can create more realistic and interactive learning environments that meet the goals of high-quality education.

Industry, Innovation and Infrastructure



Figure 2: Industry, Innovation and Infrastructure

This project explores flexible and efficient graphical segmentation solutions under GPU architecture, and promotes the innovation of graphical rendering infrastructure for industries such as VR/AR, game engines and digital content creation. The research results help strengthen industrial digitalization capacity building and support the development of emerging creative industries, which is in line with industry, innovation and infrastructure development goals.

1.6 Ethics

Risks of Misleading and False Information

High-fidelity rendering technology in Virtual Reality and Augmented Reality is prone to cause “Illusion of reality”, especially in scenes such as avatars and digital twins. Users may misjudge the authenticity of the images, or even make wrong emotional or behavioral judgments in the virtual world, affecting mental health or social cognition. There is a particular need to be wary of technologies being used to mislead, deceive or manipulate user behavior, e.g., in marketing or disinformation dissemination.

Inclusivity and Accessibility

While high-quality graphics technologies enhance the visual experience, there is also a risk that they may exacerbate the digital divide. The dependence of some advanced technologies on high-end hardware devices may result in only a few users with high-performance devices or large enterprises being able to enjoy the advantages they bring, thus limiting the popularization of the technologies and their accessibility to the general public.

To address this problem, exploring how to achieve high-fidelity visual effects on low-end devices through reasonable algorithmic degradation or approximation has become an important direction to enhance technology penetration and promote educational equity.

Chapter 2

Theory and Related Work

This chapter explain several key theories which are crucial to this thesis and provide an overview of geometry refinement techniques on the GPU, ranging from early vertex shader-based pattern instancing, hardware tessellation pipelines, to recent developments such as Micro Mesh and Mega Geometry.

2.1 Tessellation

In computer graphics, tessellation describes the process of dividing existing primitives into smaller ones thereby achieving higher realism, see Figure 3. The primitives are usually triangles or quads, depending on the tessellation algorithm used, for example Catmull-Clack [2]. The main advantage of this process is that the GPU can generate a higher density of small tuples in real-time during the rendering phase while only transmitting a coarse mesh which is a model with simplified topology and lower number of primitives counts. While enriching the detail of the model, improves the visual realism, it also decrease the bandwidth of data transfers between the CPU and the GPU.

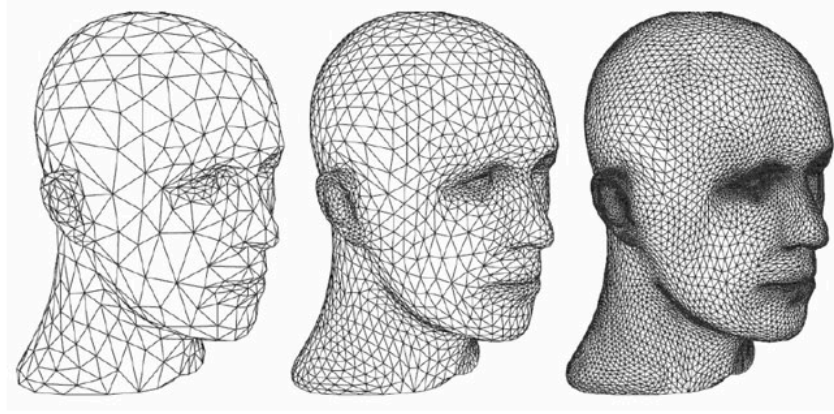


Figure 3: A human head model rendered under multiple tessellation levels.

The widespread use of surface tessellation techniques has led to the development of various derivative technologies, including GPU refinement kernels [4, 5], Hardware Tessellation [8], and GPGPU-based refinement methods [13]. These technologies enable efficient real-time model refinement, allowing the GPU to adaptively tessellate and generate the necessary smaller primitives on-the-fly during runtime. This capability has been widely applied in real-time model refinement, procedurally generated terrains, and ocean simulations, see Figure 4, providing greater flexibility and scalability in the asset creation process.

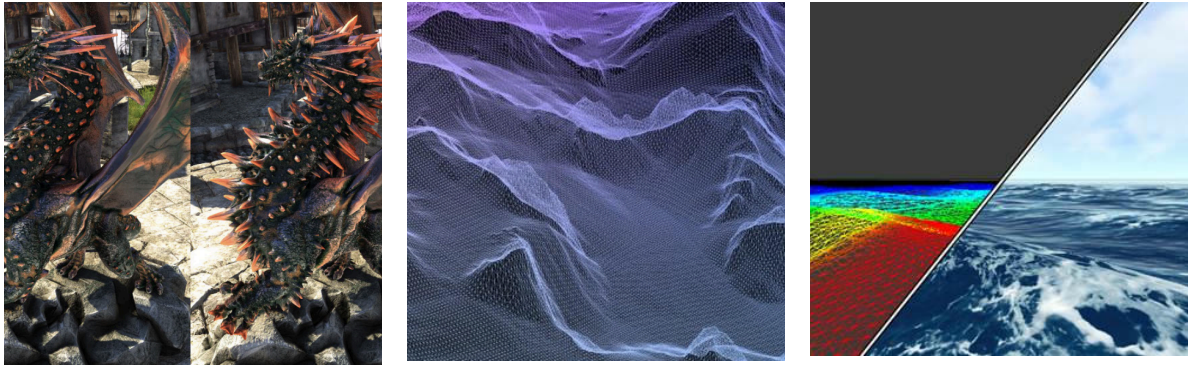


Figure 4: Dragon(left) from m Unigine Heaven Benchmark, procedural terrain(middle) and ocean (right) [14]

2.2 Geometry representation

Geometry representation is the fundamental concept in computer graphics when constructing and processing 3D models. Different model representations in different scenes can have a significant impact in terms of complexity, edibility, and rendering effects. Traditional geometric representations such as polygon, parametric surface, and subdivision surface have been widely used in modeling and rendering. However, with the continuous development of Artificial Intelligence and Machine Learning technologies, new geometric expressions, such as point cloud and neural radiance field, have emerged.

2.2.1 Topology

Topology is defined differently in different domains. In graphics geometry, topology describes the connections and organization of model data. In the real-time rendering, the topology results from a combination of points, lines and surfaces that describe the connectivity of the model, see Figure 5.

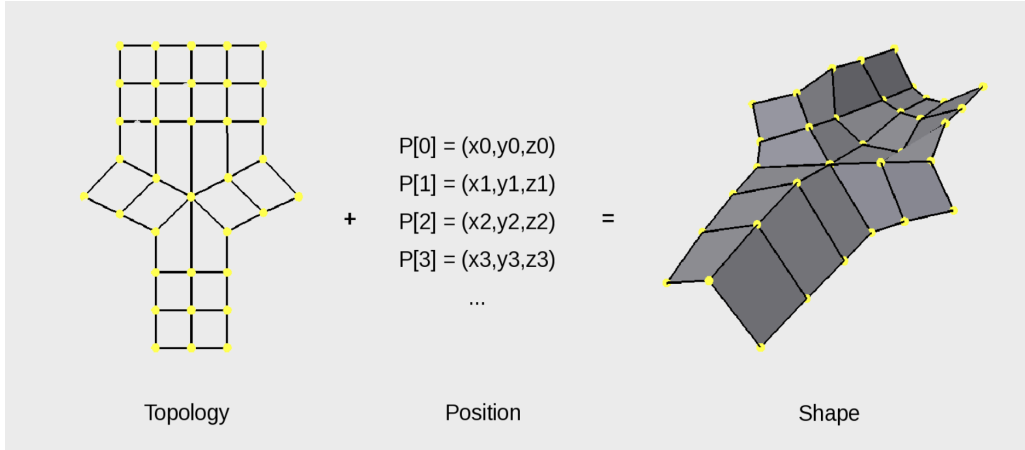


Figure 5: Topology in mesh shaping [15]

In 3D modeling, topology affects the deformation behavior of the model, the subdivision process, and even the lighting results. In subdivision algorithms [2, 3, 16], a good topology result, while reducing the generation of unnecessary vertices, also ensures the correct position and connection relationship of the generated vertices during the subdivision process. Particularly in the entertainment industry like game and movie, the topology result also directly affects the effect of animation deformation and the accuracy of physics simulation.

It is worth noting that sometimes a model that looks exactly the same on the outside may have a completely different topology behind the scenes, see figure 6.

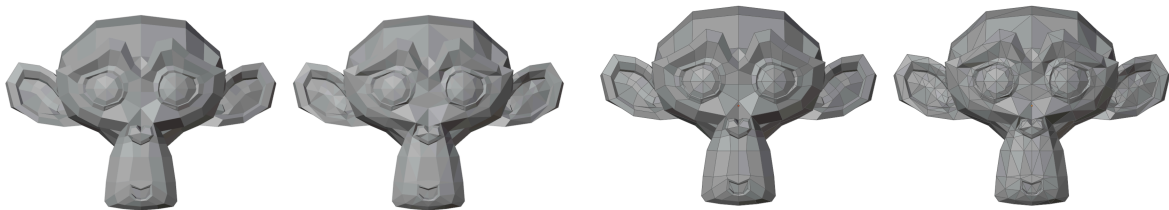


Figure 6: Same visual look(left) but different topology(right)

2.2.2 Polygon

Because triangles consist of three vertices forming a unique minimal plane, they naturally avoid coplanar problems that can occur with quadrilaterals, such as twisting or folding.

This property has enabled triangles to be established as a fundamental unit in geometric modeling and graphics rendering from the very beginning of computer graphics, and has remained a central component in the development of graphics technology in the ensuing decades. Particularly in real-time rendering, triangles have become a core graphical primitive designed into GPU architectures due to their simple data structure, the lack of ambiguity in interpolated results, and the ease of rasterization.

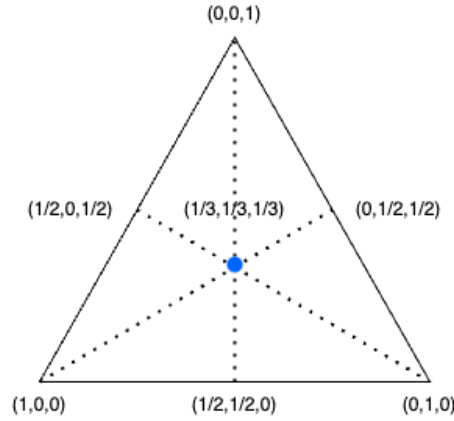


Figure 7: Barycentric coordinates of a triangle

In the GPU, vertex attributes such as position, normal or uv coordinates can be easily computed by simply using barycentric interpolation with the following formula

$$M = \alpha \cdot A + \beta \cdot B + \gamma \cdot C \quad (1)$$

which:

- M represent the interpolated attribute value
- A, B, C are the attribute at the three vertices of the triangle
- α, β, γ are the barycentric coordinates of the triangle satisfying $\alpha + \beta + \gamma = 1.0$

2.3 Compute shader

To fulfill demand for arbitrary computational task, compute shader as a shader stage, was introduced to execute massively parallel, general-purpose computation on the gpu. Unlike languages designed for gpgpu programming such as CUDA, compute shaders naturally benefit from tight integration with graphics APIs, allowing them to directly utilize graphics-related functions

2.3.1 Pipeline and Architecture

Compute shader has a very subtle relationship with the other stages in the overall graphic pipeline. The separate pipeline architecture means that the compute shader is able to operate independently of the traditional rendering pipeline, which provides flexibility but also creates a situation where While this provides flexibility, it also results in additional cost

being necessary for the compute shader to interact with the traditional rendering pipeline, which is a limitation of this article and will be explained in detail in section 6.

During the execution of the Compute Shader, the execution model is defined by Work Groups and Invocations, which control how tasks are scheduled in parallel on the GPU. Each workgroup consists of multiple threads, and each thread is an Invocation; both the workgroup and the number of threads define the layout of the computational tasks in a three-dimensional way, and the determination of the dimensions is controlled by the dimension of the data you're put in.

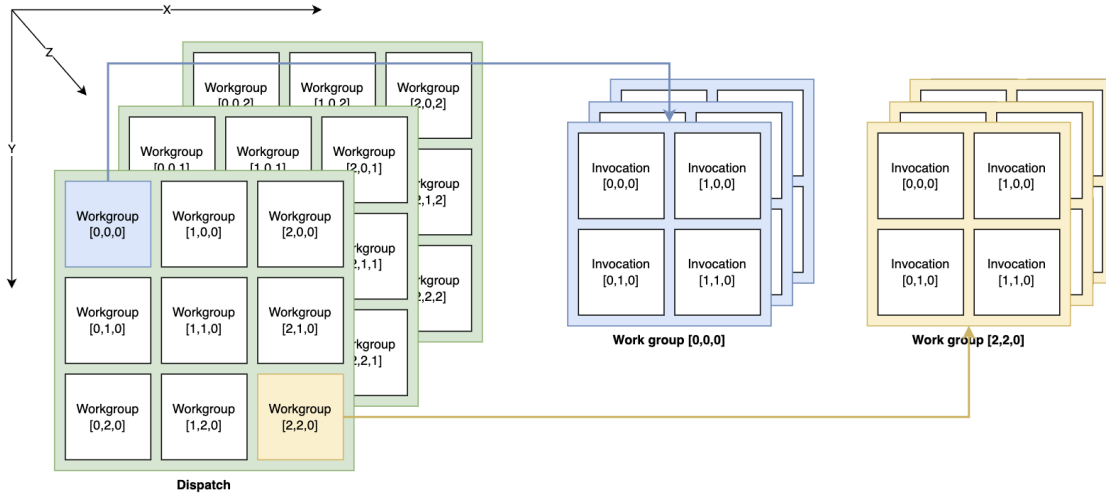


Figure 8: Compute shader execution model

The number of workgroups and the number of threads in each group determine the total number of calls to the Compute Shader. For example, dispatching work groups with dimension of (8,1,1) including (64,1,1) invocations inside one work group will invoke the compute shader $8 \times 1 \times 1 \times 64 \times 1 \times 1 = 512$ times

2.3.2 Data Access and Manipulation

In order to enable Compute Shaders to read and write data flexibly, modern graphics APIs provide mechanisms such as SSBO and Storage Image. SSBO, as a general-purpose buffer object, is commonly used to store and manipulate structured data, while storage image is used more for accessing image resources and is applicable to various tasks related to image processing. Through these two approaches, a compute shader can efficiently interact with internal and external data to support complex computation processes.

Typically, a compute shader can access data with a thread id, but during parallel execution, multiple threads may access the same memory address at the same time, especially when reading and writing to shared buffers. To avoid data contention without introducing expensive locking mechanisms, Atomic operations ensure that only one thread can read or write to a target address at a given time, thus avoiding data conflicts while ensuring parallelism.

2.4 Displacement mapping

Displacement Mapping [17] was originally a technique used to generate natural textures, and later it gained widespread application in computer graphics. As an alternative way to restore the details of the original model during real-time rendering, displacement mapping samples pre-baked or mathematically calculated vertex displacement data to modify the positions of the surface vertices of an object.

This allows for more precise detail by actually changing the shape of the object's surface, instead of just cheating the surface normals with normal maps do. Because the surface shape is really modified, displacement mapping also naturally fixes problems like self-shadowing and self-occlusion that normal or bump maps can't handle well.

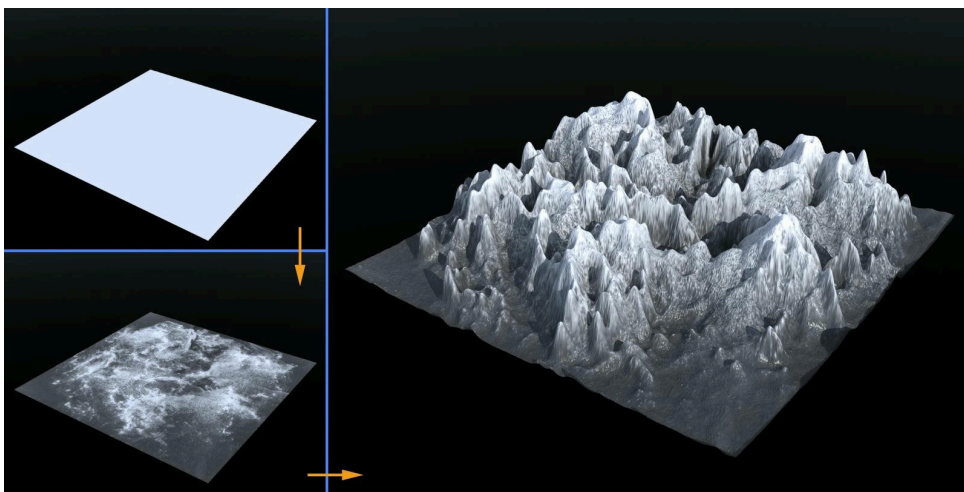


Figure 9: Displacement mapping of a quad(top left) with a height map(bottom left)

Scalar Displacement and Vector Displacement are common in most use cases where the former has a relatively simple data format, using a single-channel grayscale map to describe the displacement magnitude of each point along a certain direction—usually the model's normal direction. It is commonly used on models with simple topology, such as terrain generation see Figure 9. The latter is a more advanced representation. It records the displacement direction and magnitude of vertices in 3D space using the three RGB channels, allowing displacement in any direction: normal, tangent, or even opposite directions. Therefore, it is better suited for complex surface deformations, see Figure 10. However, the increased data size leads to higher storage and bandwidth costs, which limits its application.

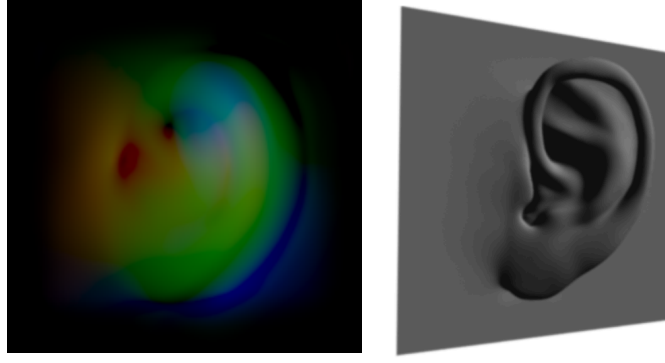


Figure 10: Vector displacement mapping from Autodesk [18]

2.5 Related work

2.5.1 Prior Refinement Schemes

Before Hardware Tessellation technique came out, Boubekeur et al. [4, 19] proposed a practice of using a vertex shader to instantiate triangle patterns later Lenz et al. [5] present an improved version with efficient data storage, and many of the concepts in this article were borrowed from these articles. They saved the required patterns for different tessellation factors in a 3D array at the cpu in advance, and uploaded these data to the gpu memory to allow the vertex shader to directly perform transformations on vertices in different patterns to achieve the generation of more primitive.

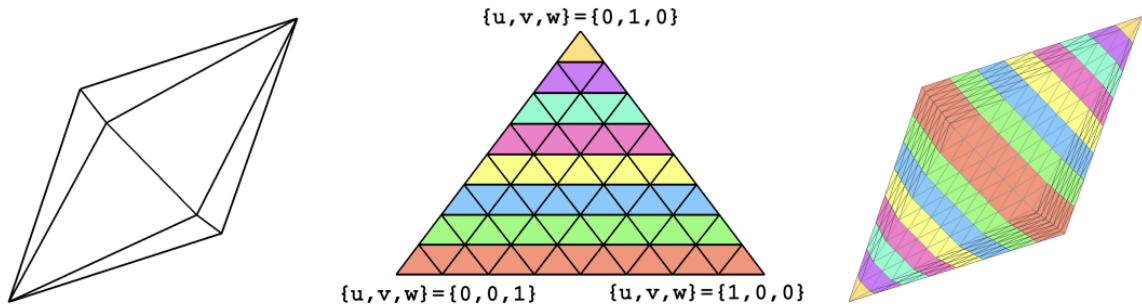


Figure 11: Coarse mesh(left), Refine Pattern(middle), Refined Mesh(right) [4]

Since this method is based on vertex shaders, there is no way for vertex shaders to communicate with each other, and only one vertex can be processed at a time. The original articles don't give the implementation details, so it's not clear how the coarse triangle and pattern are mapped.

Later, interpolation-based techniques were proposed such as Phong Tessellation [6] and PN-Triangle [7, 20], which generate smoother subdivided surfaces by interpolating

existing vertices and fitting curvature. These methods primarily aim to improve visual quality. Although they do not directly increase new topological structures, their visual effects closely resemble those of traditional geometric subdivision.

Meanwhile, GPGPU-based methods were also being developed to maximize the utilization of the GPU's parallel computing power. Schwarz et al. [13] introduce an adaptive tessellation method based on CUDA[], other GPU based tessellation methods like [21, 22] presented in recent years utilizing compute shader to generate massive amount of triangles in the GPU.

2.5.2 Hardware Tessellation

To address the CPU-GPU data transfer bottleneck associated with high-precision modeling, Direct3D 11 [23] equipped with a new hardware segmentation technology and lays the groundwork for the development of a new dynamic LODs system.

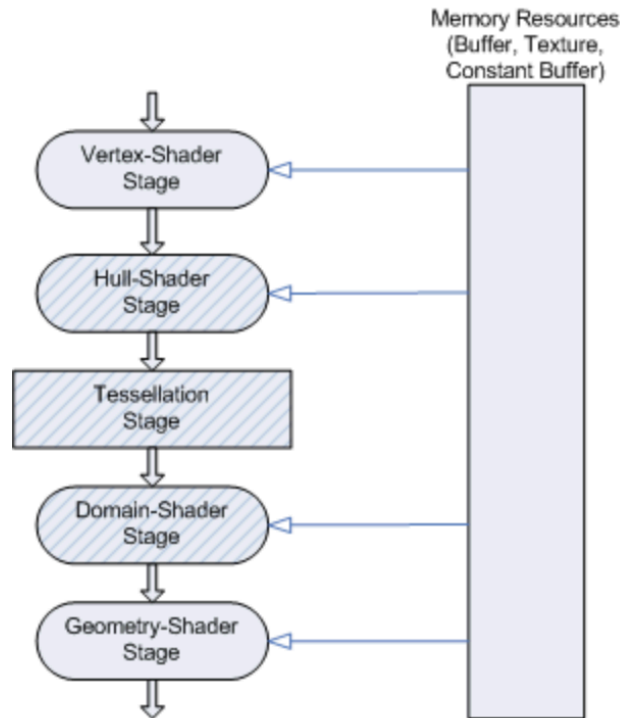


Figure 12: Hardware Tessellation pipeline from Direct3D 11 [23]

Hardware Tessellation [8] has a more flexible tess object, in the rendering pipeline these primitives are abstracted into patches, it's a collection of vertices, 3 vertices represent triangle patch, 4 represent quad patch. It achieves adaptive effect by controlling the subdivision of different edges, and supports subdivision factors with decimals and different spacing strategies to approach more detailed subdivision operations.

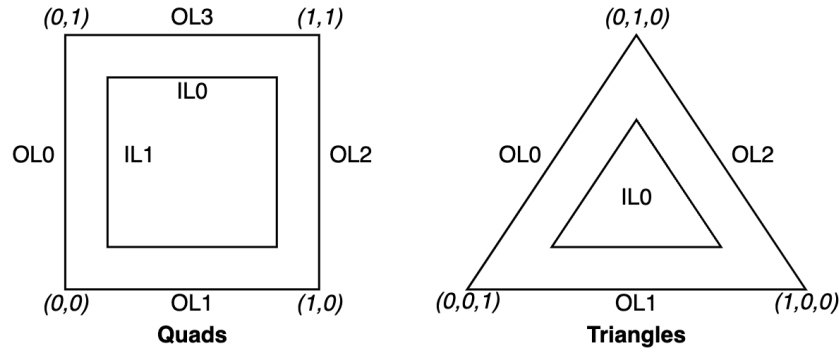


Figure 13: Common patches in Hardware Tessellation

Hardware tessellation exposes programmability through programmable shaders such as the hull shader and domain shader, offering developers a certain degree of flexibility and extensibility. This enables efficient hardware-accelerated tessellation computations while maintaining strong performance advantages.

Hardware tessellation improves real-time detail processing but has a few limitations. It operates on patches independently without shared topology, which causing duplicate vertices are generated on shared edges. Additionally, tessellation output is directly sent to the pipeline without writing to GPU memory, and since tessellation is done in a non-programmable GPU stage, manual removal of duplicates is difficult.

2.5.3 Nanite

With the release of Unreal Engine 5, Nanite [11] offers a geometry detail management approach different from traditional surface tessellation. Unlike conventional refinement schemes that use a coarse mesh and dynamically generate more triangles on the GPU to restore detail, Nanite partitions the original high-precision model into clusters—each a collection of a certain number of triangles—and dynamically loads geometry data at different levels of detail during rendering.

Nanite excels in static scenes by preserving high-detail geometry with efficient draw calls and GPU scheduling. However, it struggles with dynamic geometry like skinning or procedural deformation due to the absence of a coarse mesh. Animations require processing large numbers of high-resolution triangles, leading to heavy computational overhead.

2.5.4 Micro-Mesh and Mega Geometry

Unlike the previously mentioned refinement patterns, Micro Mesh [24–27] adopts a fundamentally different approach to reduce GPU memory usage and rendering costs. It converts a mesh into a set of compressed micro-meshes, which are encoded using barycentric coordinates and compiled into a compact binary format.

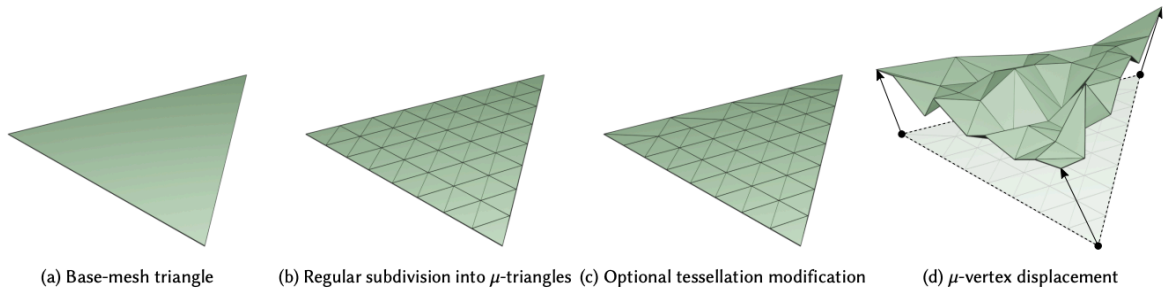


Figure 14: Micro Mesh construction

On the GPU side, NVIDIA’s modern GPU architectures [28] introduce a new Geometry Engine that provides native hardware acceleration for Micro Meshes. This allows micro-triangle data to be efficiently unpacked and traversed during real-time rendering.

However, the construction of Micro Meshes involves a series of complex preprocessing steps—such as determining tessellation rates, casting rays from each micro vertex to compute detailed vertex offsets, building acceleration structures, and compressing the generated data. As a result, Micro Meshes are primarily used in offline rendering or high-quality ray tracing tasks.

Compared to Micro Triangles, Mega Geometry [29] also adopts the concept of adaptive refinement patterns from early GPU-based methods, using a cluster-based structure for geometry management. It performs efficient dynamic tessellation on the GPU in real time through Task and Mesh Shaders, but the process is still triangle-based, leading to duplicate vertices on shared edges.

Chapter 3

Methodology

This chapter aims to introduce the methodology involved to address the research questions and challenges, including the literature study, implementation overview as well as tools and framework used.

3.1 Literature Study

The literature study has been carried out throughout the work of this project, most of the references were found through Google Scholar and the ACM Digital Library. *compute shader tessellation*, *gpgpu tessellation* are the main focus of the initial research.

Later, inspired by Mega Geometry [29] and related techniques [4, 19], the direction of the study was then shifted to its cluster based tessellation technique, the subsequent exploration focused on *gpu generic refinement schemes*, *mesh shader pipeline*, and *micro-triangle*. In addition, to deepen our understanding of the geometry representation, key words *subdivision surface* and *polygonal representations* are used. Finally to enrich the visual quality, we also studied techniques such as *subdivision surface approximation* and *displacement mapping*.

3.2 Implementation Overview

The implementation can be broadly divided into two main phases: offline and runtime. We precompute all the required triangle patterns in advance and upload them to GPU memory so that the compute shader can use them later, see Figure 15.

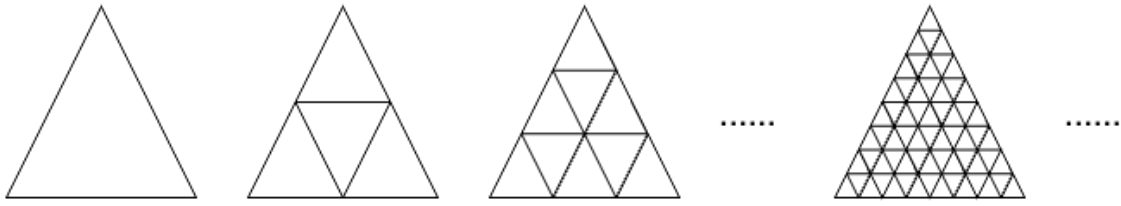


Figure 15: Pre-computed patterns

During runtime, for each visible triangle from the input coarse mesh, we select a triangle pattern based on various metrics such as a fixed tessellation rate or the distance to the camera. The selected pattern is then applied in the compute shader to generate smaller triangles. If a displacement texture is available, displacement mapping is performed afterward, see Figure 16.

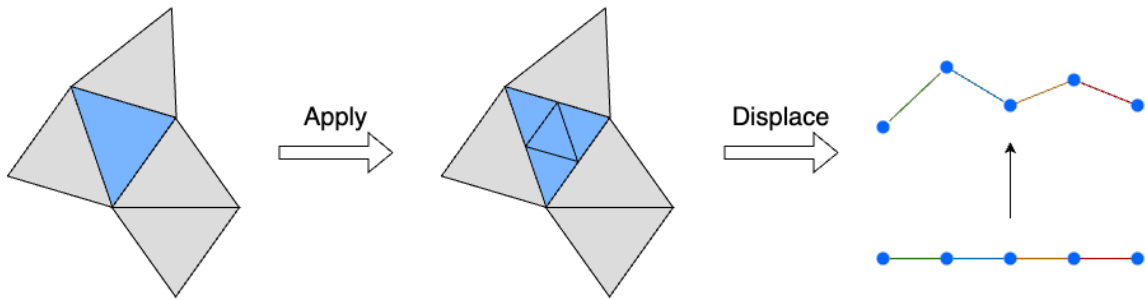


Figure 16: Short version of real time process

3.3 Tools and Framework

The development of our system is based on several key tools and frameworks. The graphics backend is built on Vulkan with the shading language GLSL. *nvpro* from Nvisia is used to fast build Vulkan application and provide several useful utilities, including convince UI framework and profiling tools. The c++ library *tinyobjloader* is used to load models in wavefront format, the open-source *stb* library is employed for image loading. *ktx2* is the primarily format of the texture used in this project to reduce GPU memory usage. For graphics debugging, we rely on Nvidia *Nsight* and *RenderDoc*. The 3D model used in this project was created in ZBrush based on the *Big Guy* character[].

Chapter 4

Implementation

In this chapter we present the implementation details of the compute shader based tessellation prototype

4.1 Pipeline Overview

Building upon the insight from the previous studies, we provide a compute shader based pipeline to demonstrate how we can leverage the GPU’s massive parallelism to reconstruct high-fidelity geometry in real time. To better illustrate the structure and functionality of our system, we present detailed explanations of several crucial stages in our pipeline in the following sections.

4.1.1 Resources Overview

This section summarizes the GPU resources used throughout the pipeline. Each resource plays a specific role in different stages of the rendering process, ranging from visibility determination to normal recalculation. Table 1 provides a detailed breakdown of these resources, including their types, usage stages, and descriptions.

Table 1: Resources Reivew

Name	Type	Stages	Description
Input Mesh data	SSBO	All Compute Stages	Mesh data of the coarse mesh
Pattern Table	SSBO	All Compute Stages	Different levels of refine pattern
LookUp Table	SSBO	All Compute Stages	Entry point to Pattern Table
Triangle Visibility	SSBO	All Compute Stages	ID of the visible triangles
Data Counters	UBO	All Compute Stages	All atomic counters
Vertex Tessellation Rate	SSBO	All Compute Stages	Tess factor of each vertex
Indirect Commands	SSBO	Indirect Setup	Setup indirect commands
Scene Information	SSBO	All Compute Stages	Scene Configuration
Frame Constants	UBO	Vertex Shader Fragment Shader	Constants for each frame
Refined Vertices	SSBO Vertex Buffer	Tessellation Stage Vertex Shader	Generated vertices
Refined Indices	SSBO Index Buffer	Vertex Shader	Generated triangles
Normal	SSBO	Fragment Shader	Normal of deformed mesh

In Table 2, we present the mesh data we used in this project.

Table 2: Input Mesh Data Reivew

Name	Triangle Counts	Vertices Counts	Edges Counts	Disk Size	Memory Size
Big guy coarse mesh	2900	1452	2900	145 KB	292 KB
Big guy detail mesh	2,969,600	1,484,802	2,969,600	241 MB	363.7 MB
Big guy displacement texture	N/A			1.2 MB	13.2 MB

4.1.2 Data-preprocessing

In the offline phase, two key components are generated: the tessellation patterns set and a lookup table that indexes them in the GPU memory. Each pattern consists of a set of vertices and triangle indices, which are stored in two large, contiguous memory blocks.

The size of each block is determined by the total number of vertices and indices across all supported tessellation levels (from level 0 to the maximum), as shown in left side of Figure 17. For example, the first three vertices in the memory block belong to the first pattern, the next six to the second pattern, and so on. The same layout applies to the index buffer.

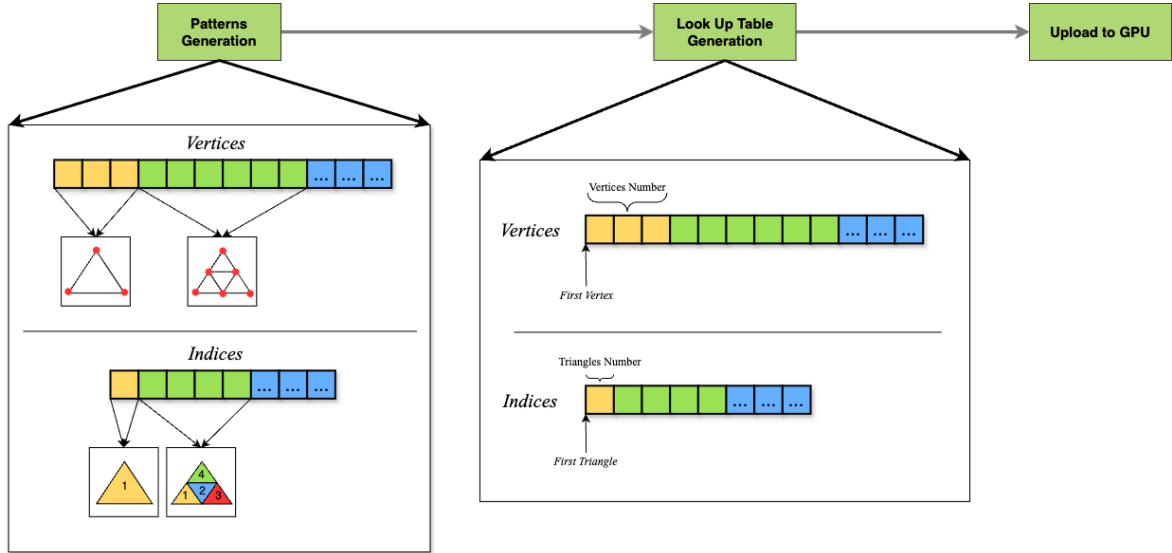


Figure 17: Offline processes

The lookup table stores the entry points for each pattern, making it efficient to access the appropriate pattern directly in GPU memory during execution. Additional information such as the indices of edge vertices on the coarse triangle is also stored to facilitate vertex reuse when applying patterns.

4.1.3 Real time framework

During the runtime phase, the system runs a sequence of compute-shader-based stages: resource cleanup, triangle visibility determination, tessellation level computation, pattern-based tessellation, and normal recalculation, see Figure 18. Since compute shaders operate within an isolated pipeline and cannot directly render to the screen, we must transfer the generated vertex and index data to the traditional rendering pipeline using SSBO, which are then bound to the vertex and fragment shaders.

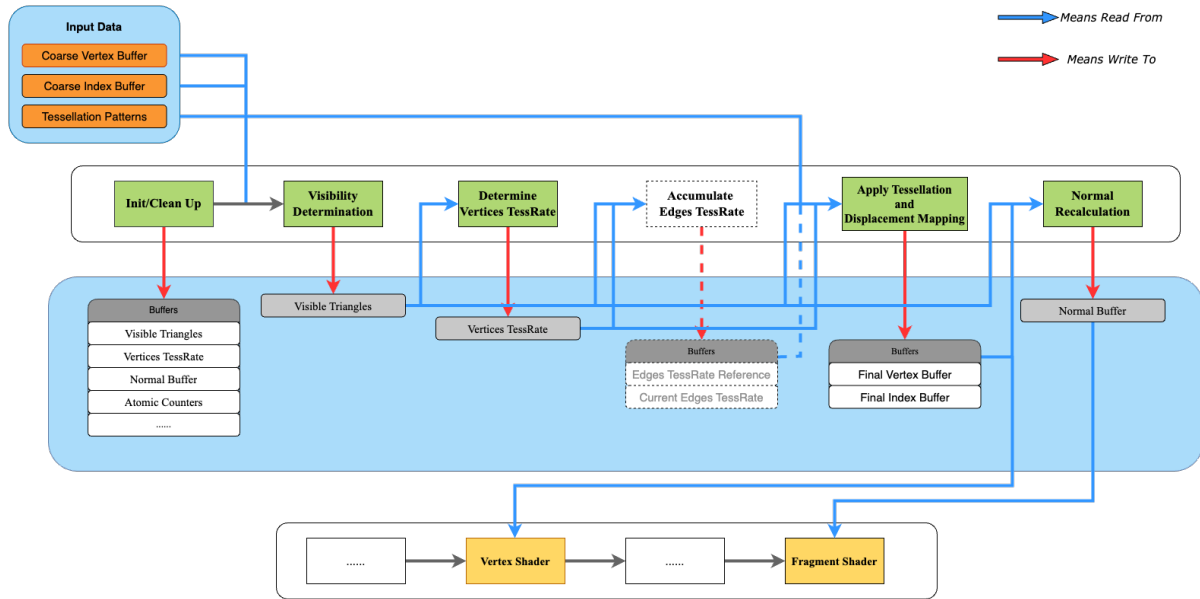


Figure 18: Realtime compute shader based framework

At the very beginning of the pipeline, several resources must be reset, see Table X, because they varied every frame, such as buffer store the triangles that are visible, atomic counter that track generated vertices and triangles, see Table X. Once the resources are properly reset, the system proceeds to determine which triangles in the input mesh are visible to the current camera, then we compute the tessellation level for each vertices from the visible triangles, based on that, the tessellation compute shader will fetch the corresponding pattern from the tessellation patterns set with the look up index, generated vertices and indices data are then written to two final buffer and subsequently bond to vertex shader to finalize the rendering.

If there are displacement textures available, vertex positions are adjusted during the tessellation stage and one additional compute pass is needed to re-calculate the normals from the the displaced vertices.

4.2 Patterns Generation

In order to achieve flexible and efficient GPU-based tessellation, this section presents algorithms for generating reusable uniform triangle patterns. With the help of these patterns, smaller, finer triangles can be quickly generated on the GPU side by simple and efficient interpolation of the barycentric coordinates.

It's crucial to note that the ordering of the vertices and triangles matters as it affects the efficiency of the subsequent vertex reuse algorithms. Therefore, the vertices and triangles in each mode are arranged in left-to-right and bottom-to-top order, as detailed in Figure 19.

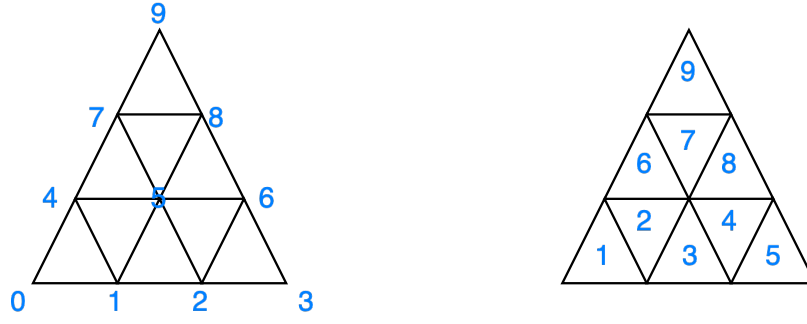


Figure 19: Vertices order(left), Triangle order(right)

For each vertex, we store its corresponding barycentric coordinates. We can divide the example pattern show in the Figure 20 into tessellation levels which 3 in the example, and each level can be regarded as a linear interpolation along u , with v increasing by 1 divided by depth, i.e. tessellation rate in each iteration. Due to the properties of barycentric coordinates and to reduce memory usage, we only store the u and v components of the barycentric coordinates for each vertex, since w can be calculated with $w = 1.0 - u - v$.

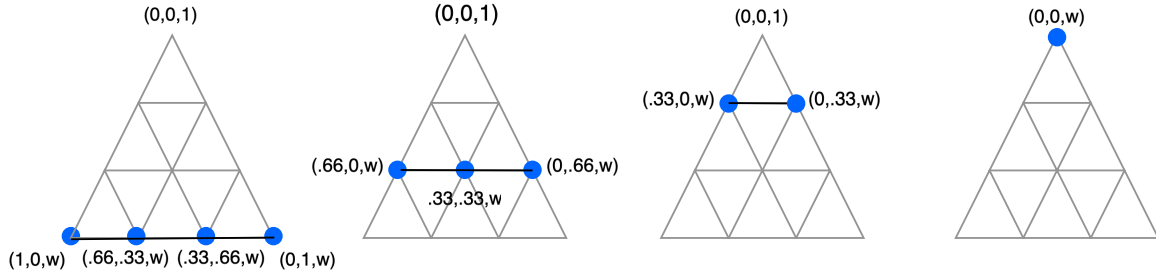


Figure 20: Vertices interpolation with gradually increase depth value

Algorithm 1 Pattern Vertices Generation

```

1: function PATTERNVERTICES(max_tess_rate, pattern_vertices)
2:   idx  $\leftarrow$  0
3:   cur_tess_rate  $\leftarrow$  1
4:   for cur_tess_rate  $\leq$  max_tess_rate do
5:     i  $\leftarrow$  0
6:     j  $\leftarrow$  cur_tess_rate
7:     for i  $\leq$  cur_tess_rate do
8:       for j  $\geq$  cur_tess_rate - i do
9:         fracs  $\leftarrow$   $\frac{i}{\text{cur\_tess\_rate}}, \frac{j}{\text{cur\_tess\_rate}}$ 
10:        bary_coord  $\leftarrow$  Lerp[fracs]
11:        pattern_vertices[idx]  $\leftarrow$  bary_coord
12:        idx  $\leftarrow$  idx+1
13:        j  $\leftarrow$  j-1
14:      end
15:    i  $\leftarrow$  i+1

```

```

16:   end
17:   cur_tess_rate ← cur_tess_rate + 1
18: end
19: end

```

To efficiently build reusable Uniform Patterns, we precompute the triangle topology, i.e. index order of each pattern. We make the current triangle a quadrilateral by adding a virtual triangle along its right side, so that two simple loops can compute the order we want, see Figure 21. We use the depth of the pattern as the loop variable.

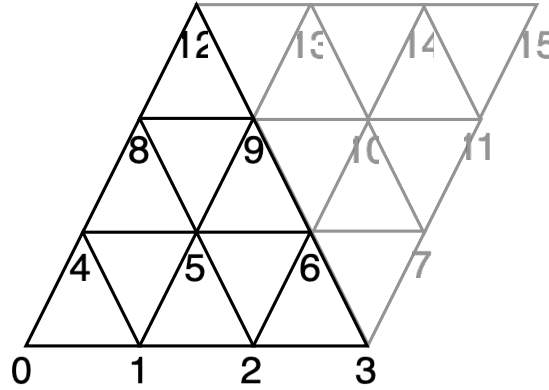


Figure 21: A example pattern for computing triangle indices

We use the depth of the pattern as the basic for the loop, starting at the bottom, and constructing the triangles in between, using ID information about the vertices of the current layer and the next layer in turn. To avoid unnecessary memory traversal and index out of bounds, we don't need the extra vertex information on the virtual triangles, so we need to calculate the maximum index value for each layer by using the partial sum of an arithmetic sequence and the depth value, and give up on generating the index of the current triangle when there are vertices exceeding the maximum index value computed by formula (2) for the corresponding layer.

$$S_n = \frac{n * (a_n - a_1)}{2} \quad (2)$$

Algorithm 1 Pattern Indices Generation

```

1: function PATTERNVERTICES(max_tess_rate, pattern_indices)
2:   idx ← 0
3:   prev_row_len ← 0
4:   cur_tess_rate ← 1
5:   for cur_tess_rate ≤ max_tess_rate do
6:     depth ← cur_tess_rate
7:     j ← 0
8:     for j ≤ depth do
9:       max_idx ←  $\frac{(j+2)*((cur\_tess\_rate + 1)*(cur\_tess\_rate - j))}{2} - 1$ 
10:      num_row_quad ← cur_tess_rate - j
11:      cur_row_len ← cur_tess_rate + 1 - j
12:      i ← 0

```

```

13:   for  $i < \text{num\_row\_quad}$  do
14:      $\text{start\_point} \leftarrow i + \frac{j * (\text{cur\_tess\_rate} + 1) + (\text{cur\_tess\_rate} - j)}{2}$ 
15:      $\text{fir\_point} \leftarrow \text{start\_point}$ 
16:      $\text{sed\_point} \leftarrow \text{start\_point} + 1$ 
17:      $\text{thi\_point} \leftarrow \text{start\_point} + \text{cur\_row\_len}$ 
18:      $\text{pattern\_indices}[\text{idx}] \leftarrow \text{vec3}(\text{fir\_point}, \text{sed\_point}, \text{thi\_point})$ 
19:      $\text{idx} \leftarrow \text{idx} + 1$ 
20:      $\text{fir\_point} \leftarrow \text{start\_point} + 1$ 
21:     if  $\text{fir\_point} + \text{cur\_row\_len} > \text{max\_idx}$  then
22:        $\text{sed\_point} \leftarrow 0$ 
23:     else
24:        $\text{sed\_point} \leftarrow \text{fir\_point} + \text{cur\_row\_len}$ 
25:     end
26:   end
27:   if  $\text{sed\_point} == 0$  then
28:     continue
29:   end
30:    $\text{thi\_point} \leftarrow \text{start\_point} + \text{cur\_row\_len}$ 
31:    $\text{pattern\_indices}[\text{idx}] \leftarrow \text{vec3}(\text{fir\_point}, \text{sed\_point}, \text{thi\_point})$ 
32:    $\text{idx} \leftarrow \text{idx} + 1$ 
33:    $i \leftarrow i + 1$ 
34: end
35:    $j \leftarrow j + 1$ 
36: end
37:    $\text{cur\_tess\_rate} \leftarrow \text{cur\_tess\_rate} + 1$ 
38: end
39: end

```

Using the above method, we can efficiently pre-generate triangle uniform patterns at arbitrary resolutions on the CPU side to support high-concurrency tessellation on the GPU. For detailed pseudocode, please refer to Algorithm 1 and Algorithm 2.

4.3 Visibility and tess rate determination

In addition to resource initialization, the formal execution of a real-time rendering pipeline begins with two key steps: rejection of non-visible triangles and surface tessellation rate assignment. Discarding invisible triangles at the initial stage of the pipeline is essential because these triangles do not contribute to the final result in subsequent computations, but consume valuable computational and memory resources, resulting in wasted performance.

To this end, we employ a strategy similar to fix culling function in the graphics hardware, i.e. back culling to determine the visibility of triangles. Specifically, we first construct a face normal for each triangle and compute the dot product between it and the camera view direction. If the dot product is greater than zero, the triangle is facing the camera and is visible; conversely, if the dot product is less than or equal to zero, the triangle has its back to the camera and should be culled. This approach is both simple and efficient, and is especially suitable for parallel computing environments.

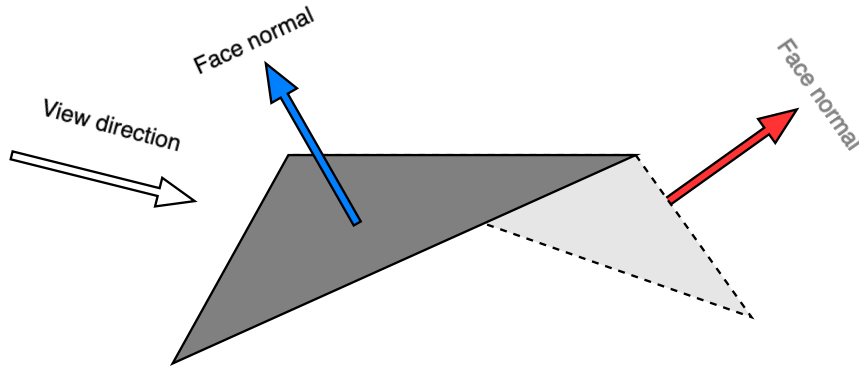
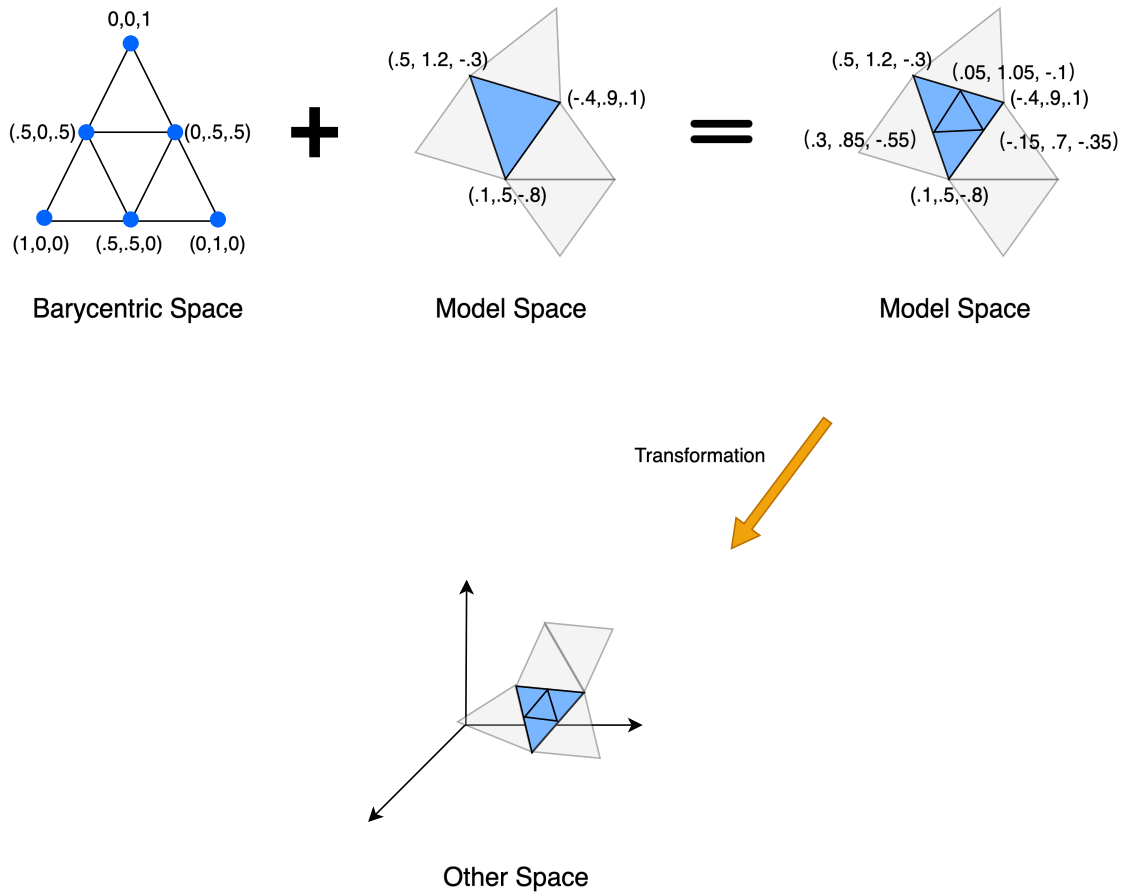


Figure 22: Culling

However, the focus of this paper is on the design of the rendering pipeline framework based on the Compute Shader. In order to simplify the implementation and avoid the T-Junction[] problem caused by the inconsistency of the tessellation levels of neighboring triangles, we choose to uniformly assign the same level to all vertices. The possible impact of this strategy and how we further deal with the T-Junction problem will be discussed in detail in the following sections.

4.4 Tessellation and Displacement

After the preparatory work, the actual execution of Tessellation itself is not complicated. The core of Tessellation is to use the barycentric coordinates of the pre-generated pattern and the vertex data of the original coarse triangle to calculate the real vertex positions after refinement by the interpolation formula (1).

**Figure 23:** Tessellation process

To further enhance geometric detail, displacement mapping can be applied on top of the interpolated surface. Once the base position is computed through barycentric interpolation, we offset it along the original surface normal using a displacement value sampled from a displacement texture. This value is typically fetched using the corresponding interpolated texture coordinates.

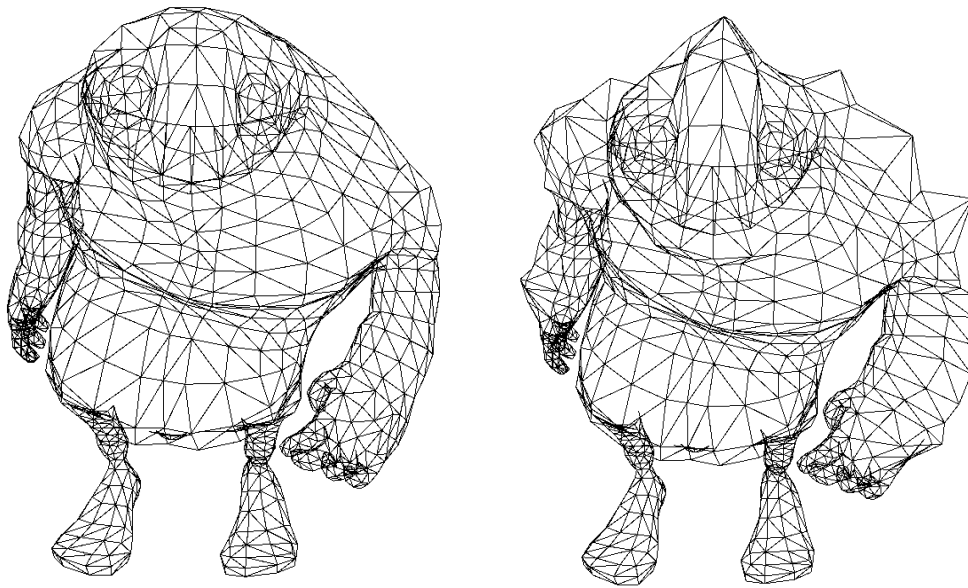


Figure 24: Displacement Mapping

4.4.1 Vertices Deduplication

TODO

4.5 Normal Re-Calculation

In the case of dynamic water or procedurally generated terrain, where vertex positions are offset by mathematical functions or noise functions such as Perlin noise[], normals can also be directly calculated from the corresponding mathematical expressions. Even for terrain, whose normals naturally point mostly upwards, reasonably accurate lighting results can still be achieved without extensive processing.

However, when dealing with complex 3D models, the situation is different. If we don't rely on a normal map, we need to recalculate the displaced normals in some other way.

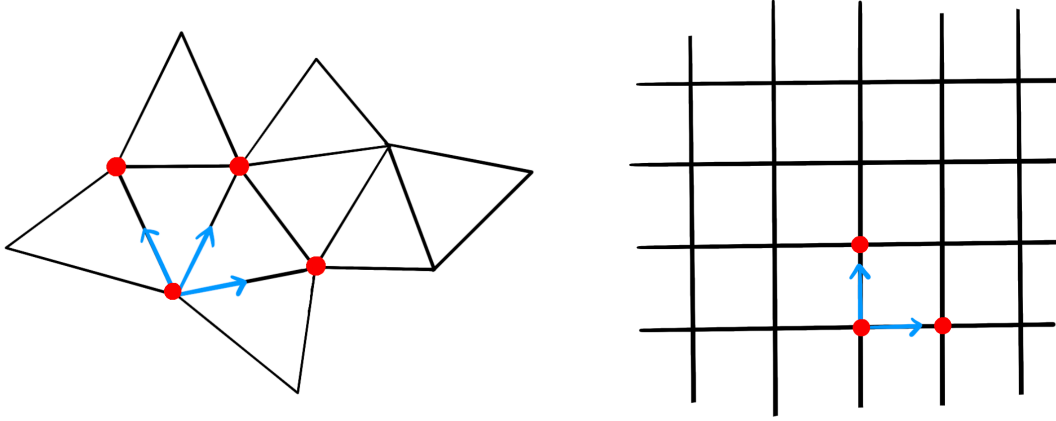


Figure 25: Quad has better topology to calculate derivatives than triangle

If only face normal is used, each triangle will appear distinctly flat and the overall effect will be stiff. Therefore, it is more desirable to compute normals for each vertex, so that during the rasterization phase, the GPU can interpolate the normals inside the triangles, resulting in a smoother surface effect. Specifically, for each tessellated triangle, we first construct its face normals and accumulate them to each of the three vertices constituting the triangle. Taking the topology in Figure 27 as an example, if a vertex X is connected to multiple triangles, then that vertex will receive the face normals from each neighboring triangle. In this way, the final normal of vertex X is jointly determined by the normals of all its neighboring triangles, providing better smoothing.

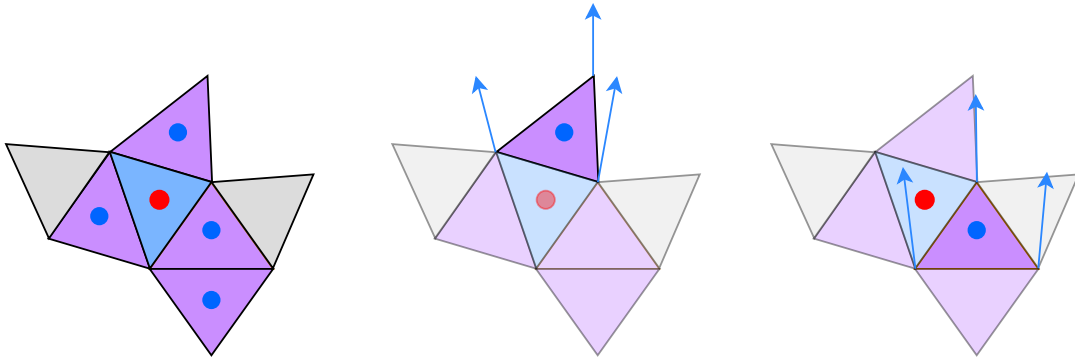


Figure 26: Affected region of normal recalculation on the central triangle(left) and normal accumulate process(mid and right)

Since this process is executed in parallel in the Compute Shader, the final normals of all vertices are not immediately available until the end of the calculation. Therefore, these normals need to be normalized to ensure they are in the correct range before they are subsequently used.

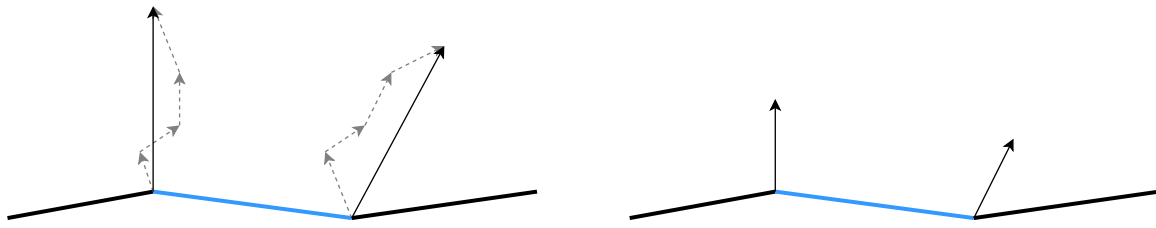


Figure 27: accumulated normals(left) and normalized normal(right)

Chapter 5

Evaluation and Results

In this chapter, we present a comprehensive evaluation of our pattern-based compute shader tessellation framework. We adopt a combination of quantitative and qualitative criteria, including most of the stages in the pipeline. In terms of performance evaluation, we compare our approach against existing methods including hardware tessellation, other GPU tessellation schemes and virtualize geometry system Nanite.

5.1 Visual Quality Analysis

The evaluation of visual quality mainly focuses on geometric fidelity, attribute coherence, shading, and visual consistency

5.1.1 Visibility and Tessellation

Here in Figure 28 and 29, we show how the same tessellation pattern with level 10 can be reused across models with completely different topologies. Despite variations in vertex connectivity and surface curvature, the pattern-based method maintains geometric correctness and visual consistency.

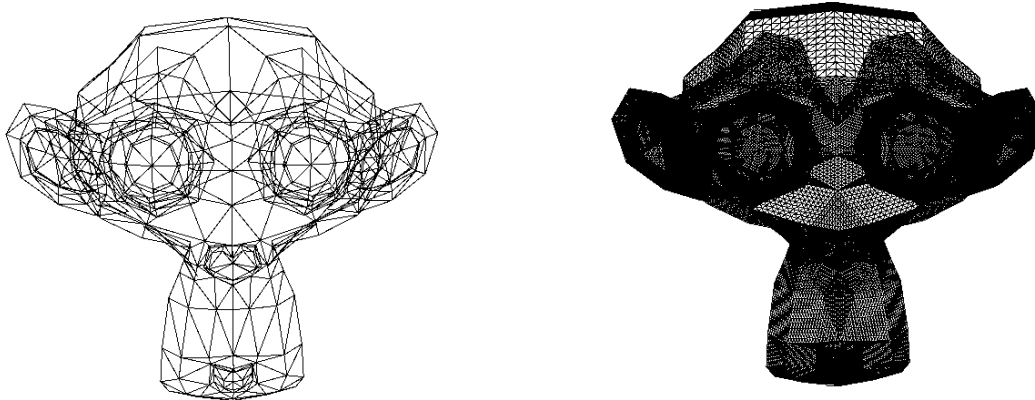


Figure 28: Suzzane coarse mesh(left) and tessellated Suzzane mesh

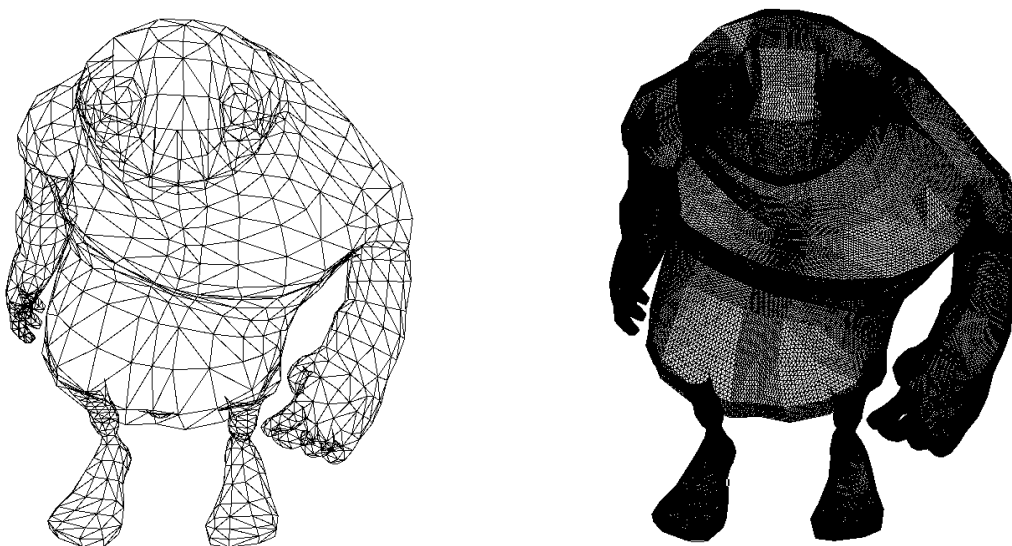


Figure 29: Big guy coarse mesh(left) and tessellated Big guy mesh

But we definitely not satisfied with only 10 tessellation level, the following image demonstrate what is the model looks like with pattern 100 tessellation level.

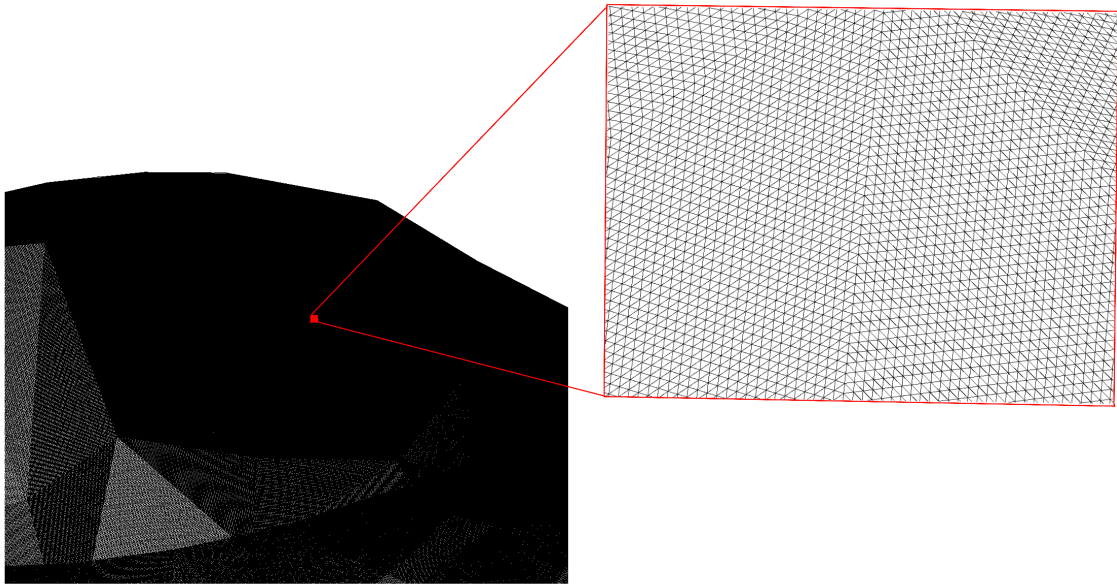


Figure 30: Big guy with 100 tessellation level

Figure 31 compares the impact of culling operations on visibility at a fixed camera view. As can be seen in the second image, even a simple culling algorithm can help us save almost 50% of the computation and memory overhead.

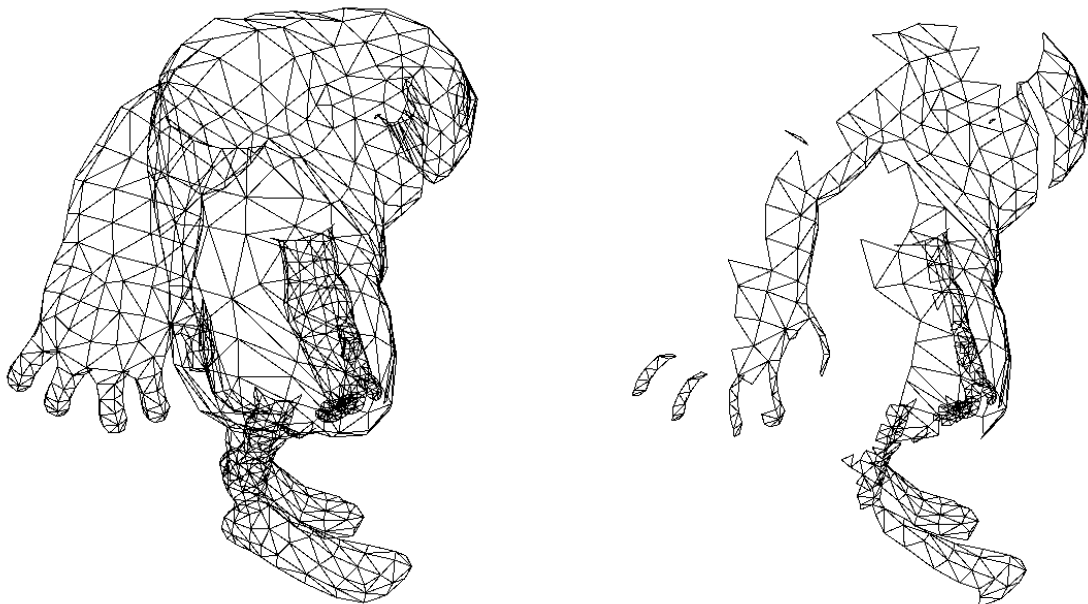


Figure 31: Side view of disable culling(left) and enable culling(right)

5.1.2 Displacement mapping

Tessellation and displacement mapping have always played an important role in procedural terrain generation, and our method can also be applied to common terrain generation scenarios, see Figure 32.

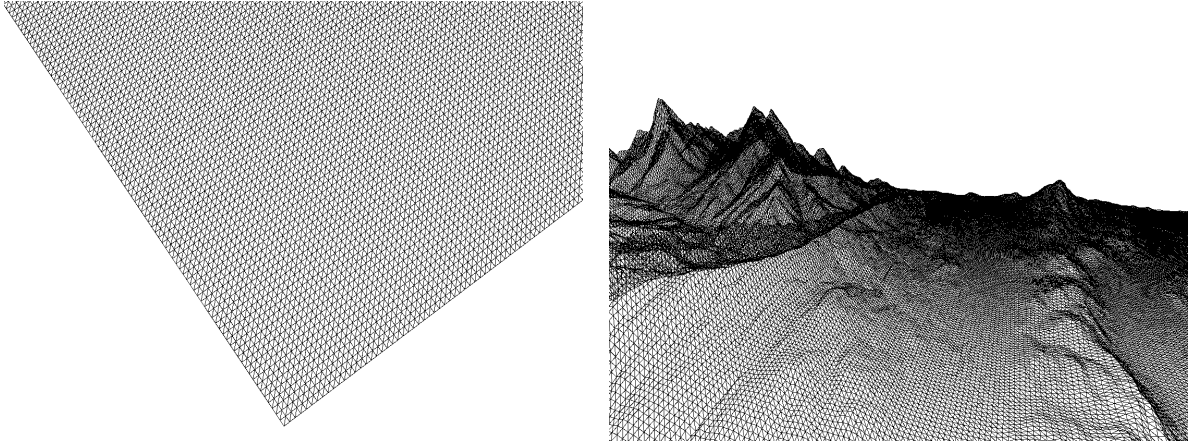


Figure 32: Terrain with 50 tessellation level(left) and then apply displacement mapping

To evaluate the generality of our method, we further apply displacement to complex 3D models. Figure 33 shows that, even with arbitrary topology, our framework effectively refines the surface, extending beyond traditional terrain applications.

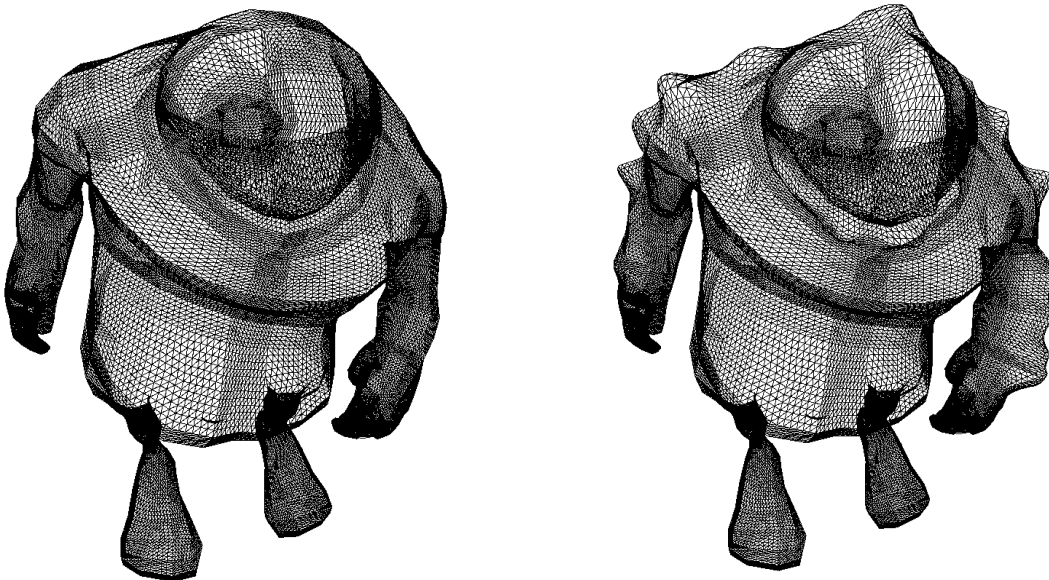


Figure 33: Big guy with 10 tessellation level(left) and then apply displacement mapping

5.1.3 Normal Recalculation

In Figure 34, we can see that the lighting calculation after displacement is not particularly accurate.

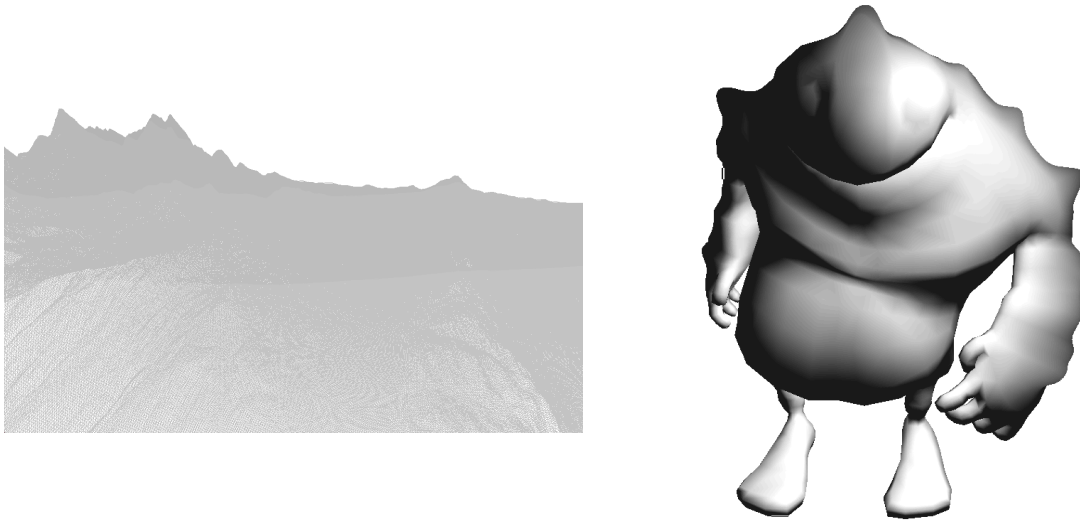


Figure 34: Lighting results with original terrain normal(left) and model normal(right)

This is because we have not recalculated the normals of the displaced vertices. Generally speaking, when a mesh is deformed, the curvature of the surface changes, and so do the corresponding tangents and normals, see Figure 35. While this may not be obvious on a terrain, the results in Figure 36 show that it is easier to see on more complex 3D models. visible on more complex 3d models.

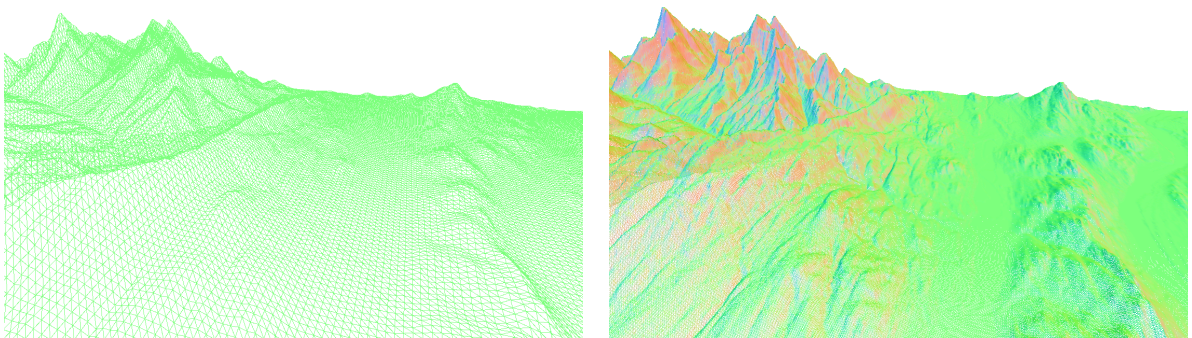


Figure 35: Terrain with input normal(left) and re-calculated normal(right) after displacement mapping

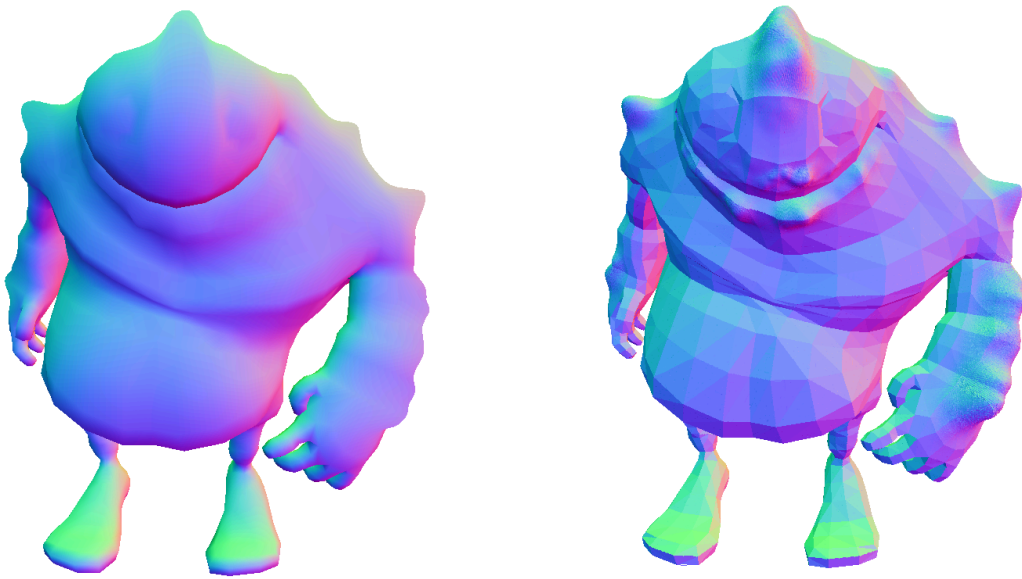


Figure 36: Big guy with input normal(left) and re-calculated normal(right) after displacement mapping

Figure 37 demonstrates the impact of different normals on the lighting of the 3D model.

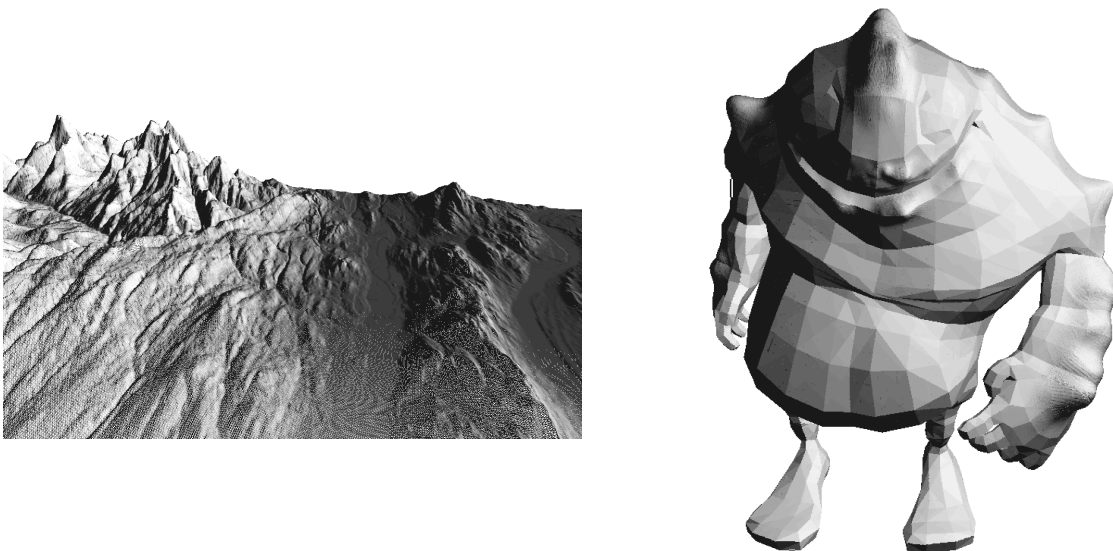


Figure 37: Lighting results with re-calculated normals

5.2 Performance Analysis

As we mentioned in the previous chapter, the compute shader pipeline also incurs some additional performance overheads. The following table and figure describes in detail the overhead incurred in our framework.

Table 3: Resources in GPU Memory

Buffer Name	Size in memory
Triangles Visibility	0.285 MB
Refine Patterns	5.47 MB
Indirect Commands	160 B
Refined Vertices	403 MB
Refined Indices	403 MB
Scene Configuration	64 B
Re-calculated Normal	403 MB
Frame Constants	176 B

It's worth noting that most of the memory here is pre-allocated in order to meet the needs of dynamically generated vertices. In the following analysis, we focus solely on the actual memory used by the generated geometry, excluding the pre-allocated buffer space reserved for dynamic vertex generation.

Following figure show the GPU time of each stage in millisecond with the tessellation factor of 50.

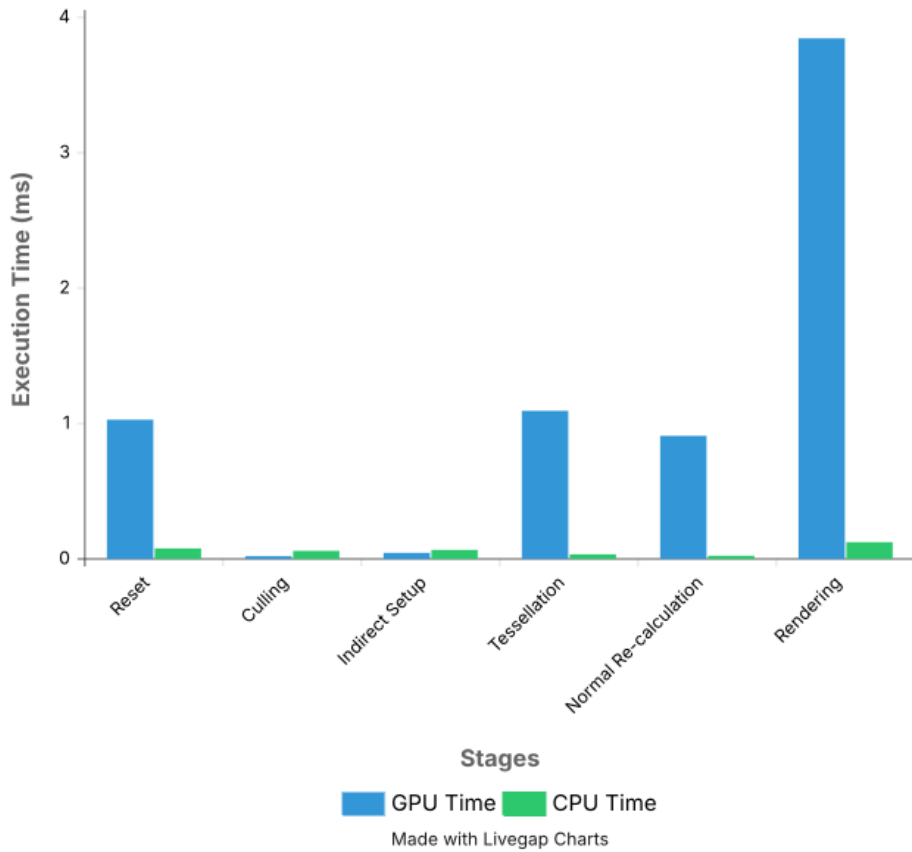


Figure 38: GPU and CPU execution time of different stages

While documenting the performance overhead of our own framework is necessary, in order to fully assess the utility of the approach, we must also compare it to several existing mainstream schemes. In the next section, we will show the performance results in turn against state-of-the-art techniques such as direct rendering of highly detailed models, Hardware Tessellation, and Nanite.

Compare to original detailed mesh

In Table 2, we present comprehensive information about the input detailed mesh data. This section we present the subsequent analysis focusing on model loading time, rendering time and memory consumption. To get the same triangle amount, input coarse mesh will apply pattern with tessellation level 32 to generate $2900 * 32 * 32 = 2969600$ triangles.

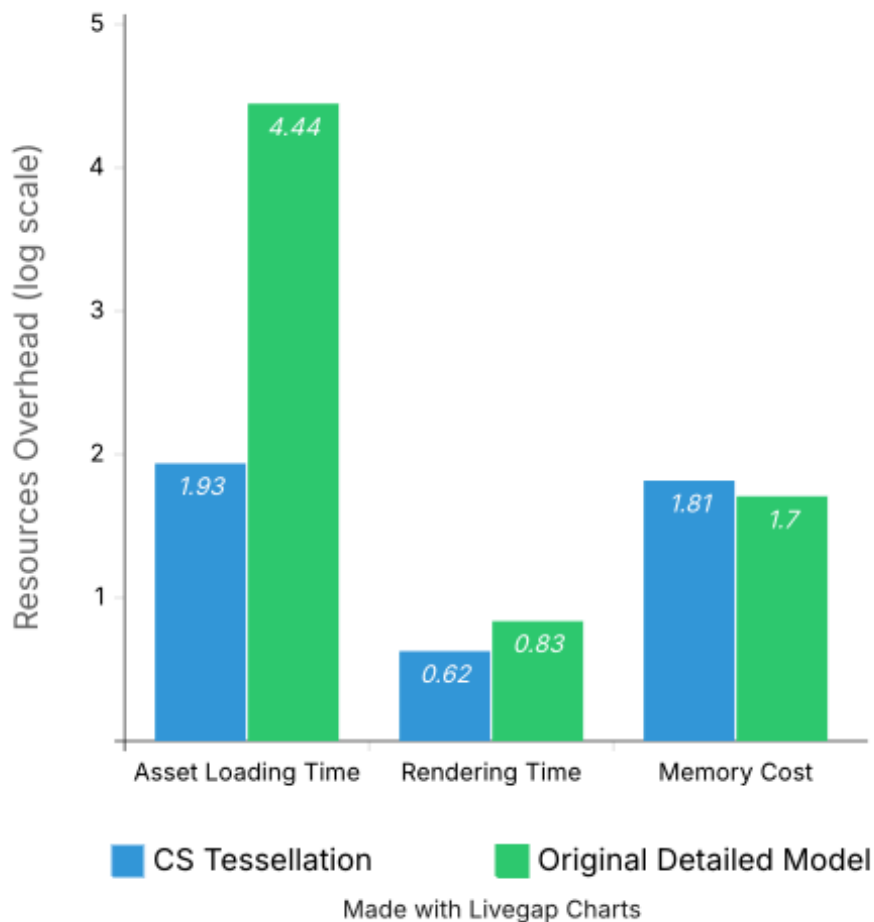


Figure 39: Comparison between coarse mesh with compute shader tessellation and original detailed model

Since the generated vertices must inevitably be written back to GPU memory, the actual memory usage remains roughly the same — or even slightly higher due to the additional storage required for the displacement texture. However, the model loading time is significantly reduced from 27,820 ms to 86 ms, and the rendering performance also improves by approximately 37%, with the rendering time decreasing from 7.3 ms to 4.61 ms.

Compare to Hardware Tessellation

Since Hardware Tessellation directly stream generated primitive data to the GPU cache, although it is possible to estimate memory usage depending on the amount of generated vertices and triangles, we only consider the rendering consumption as hardware implementation detail such as data compression is unknown.

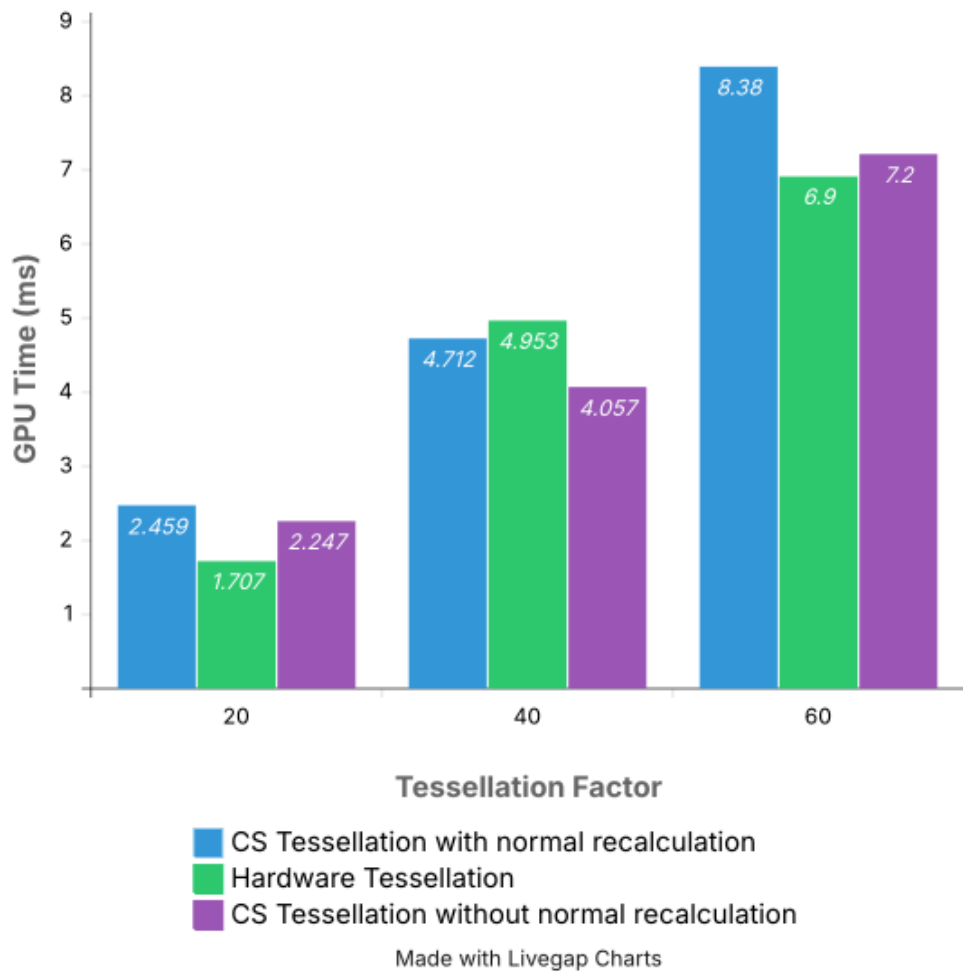


Figure 40: Comparison between compute shader tessellation with hardware tessellation in different tessellation factors

The figure above compares the rendering performance of the two approaches using the same tessellation factors. As we can see, at moderate tessellation levels, the compute shader-based tessellation shows slightly better processing times. When the tessellation factor is either too high or too low, hardware tessellation performs slightly faster.

Chapter 6

Limitation and Future work

This chapter discuss the challenges and issues we meet throught out the whore precess period. We also outlined several possible solution or related research for the furture work

6.1 Data Optimization

GPUs have relatively limited memory resources available compared to host systems. In this memory-constrained environment, any redundant data storage can negatively impact system performance.

Although it was mentioned earlier that we only store u and v in the barycentric coordinates of each vertex, they are currently each still stored as 32-bit floats, which means that they take up 8 bytes per vertex. To further consolidate their memory footprint, we can encode v as 16-bit integers respectively, combining them into a single 32-bit, i.e. 4 bytes data structure, thus halving the memory overhead per vertex. Similarly, different bit-widths can be used for the storage of triangle indices [29].

Besides the compression strategies mentioned above, the first paper, AMD proposes a lossy compression format for small meshlet patches by quantizing vertex coordinates and encoding topology compactly, achieving efficient and hardware-friendly geometry compression [30]. [31] focuses on triangle strips, optimizing rendering efficiency by reordering vertex sequences and constructing the longest possible strips to reduce redundant vertex transmission. [5] improve the pattern-based mesh refinement method [4], storing only essential refinement patterns and local information, significantly reducing the storage and transmission overhead of refined meshes while enabling efficient GPU parallel processing.

6.2 Adaptive Patterns

The patterns we have used so far are uniform patterns, i.e., the same tessellation rate is applied to all three sides of the triangle. Although this approach is simple to implement and to some extent can avoid the crack problem caused by T-junction by uniform tessellation rate, it still has many limitations in practical application.

From the perspective of rendering efficiency, different regions often have different geometric complexity or viewpoint importance. Uniform tessellation is similar to the common discrete LOD method in games, which is unable to flexibly adjust the tessellation density of each region according to the difference in geometric complexity or viewpoint, thus introducing a large number of redundant vertices. Compared to adaptive patterns [19, 29] based on curvature or viewing angle, uniform patterns generate too many invalid vertices in flat areas or areas far away from the viewing angle, adding unnecessary rendering and memory overhead.

6.3 Generic Vertices Deduplication

Based on my limited research and investigation, I found that no matter in the latest mega geometry [29], or slightly earlier gpu tessellation [8, 13, 21], or even hardware tessellation, they will inevitably generate duplicate vertices on the shared edge, because they are single triangle or quad as the object of tessellation, and do not consider the overall topological information. This results in a adjacent triangle suddenly becoming two separated triangles, with overlapping vertices on their shared edges, see Figure 41.

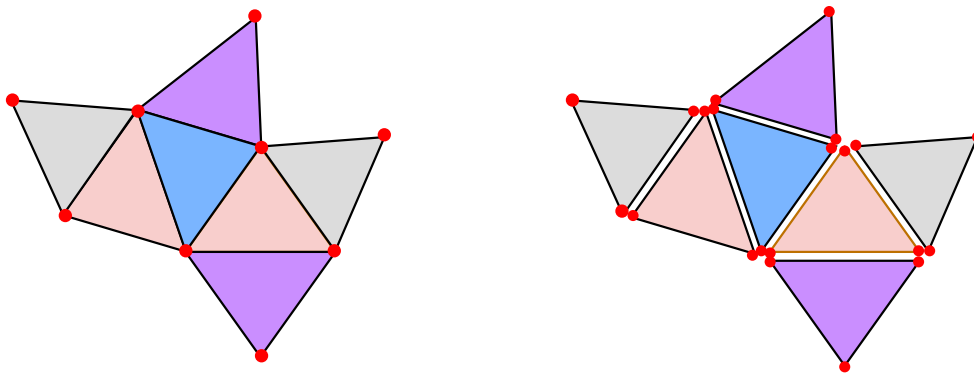


Figure 41: Adjacent triangles(left), Separated triangles(right)

That's why when I recalculate the normal, it will cause the face normals of adjacent triangles contribute to the same shared vertex to ensure smooth normal interpolation now only affects the three vertices explicitly forming the current triangle due to the presence of duplicated vertices, each face normal, see Figure 42.

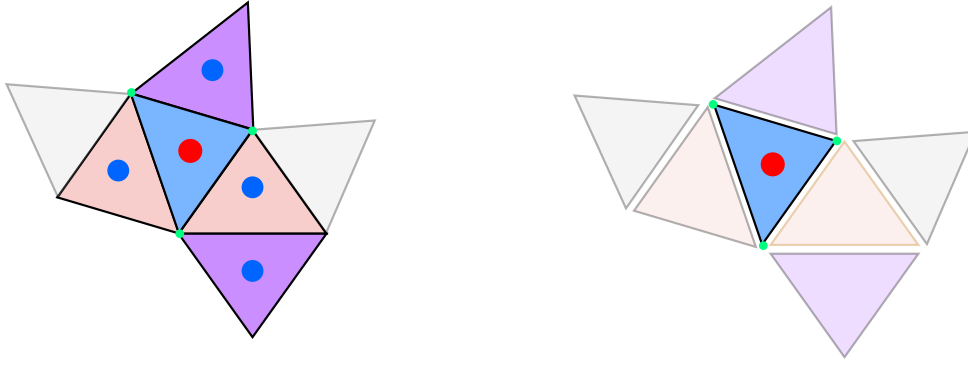


Figure 42: Triangles involved normal accumulation before tessellation(left) and after tessellation(right)

If the curvature of the model is too large, then two neighboring triangles will have a large converging normal direction, and the shared edges of the two triangles will start to fight for the right to render this edge, because they appear to be one edge, but they are actually composed of different but overlapped vertices, see Figure 43.

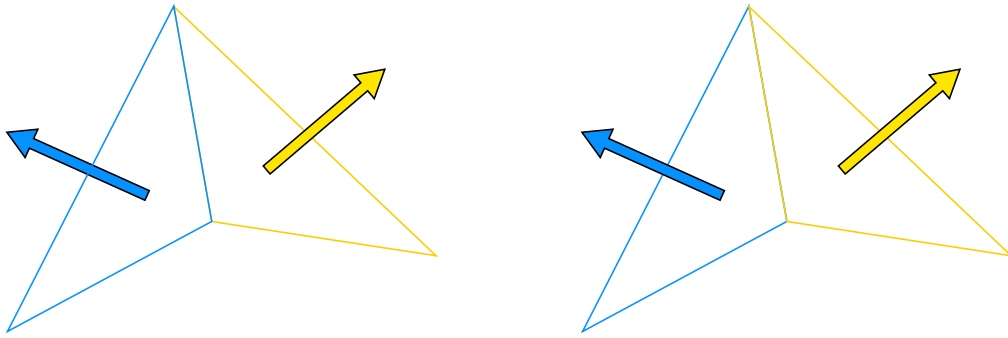


Figure 43: Fighting normal

But causing such a visual flaw is not the worst result, the gpu memory consumed by generating redundant vertices is what we need to optimize more. In order to minimize the redundant vertices generated during tessellation, there is no very generic solution in the academic world so far, and since deduplication has a highly sequential nature, it is actually more suitable for CPU execution, and removing the redundant vertices in parallel in a compute shader is a relatively difficult task.

Wald et al. [32] present a highly parallelized vertices removal algorithm using cuda was proposed, but cuda is a different language, and forcing it to manipulate the data generated by the compute shader would introduce many additional operations such as memory mapping.

As shown in the Figure 44, A pattern with tessellation level 10 results in approximately half of the vertices being duplicated. Although the proportion of duplicated vertices decreases as the tessellation factor increases, the absolute number of duplicate vertices continues to grow.

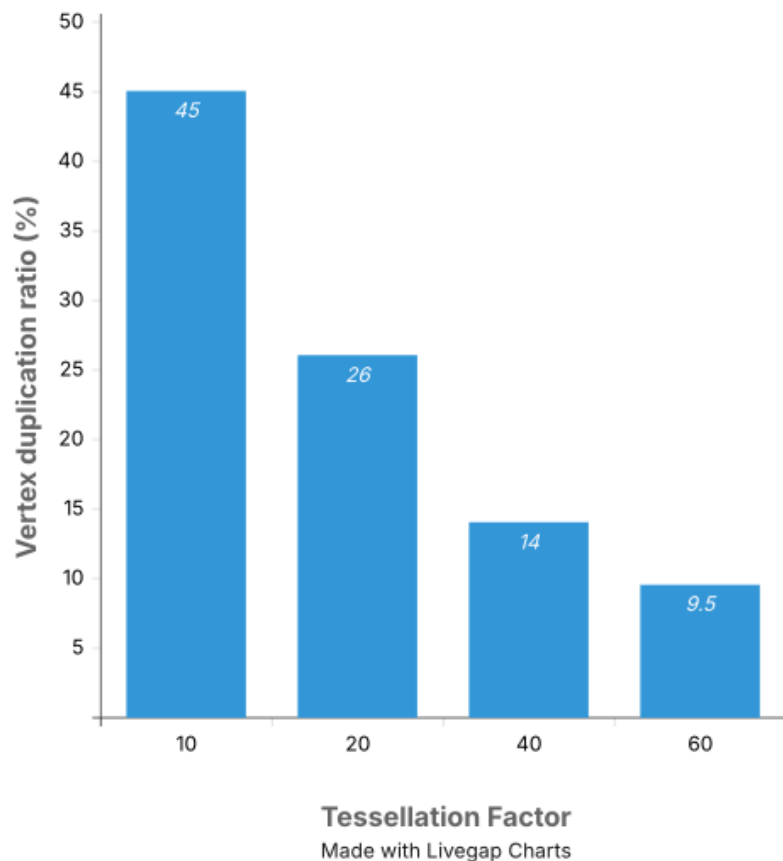


Figure 44: Proportion of duplicate vertices in each tessellation pattern

However, the duplicate vertices removal strategy will be differ from different tessellation strategy. For example, if the tessellation is a wild range tessellation, say based on the whole cluster, the duplicate vertices to be removed will only exist at the edges of the cluster. As a result, providing a universal solution for duplicate vertex removal remains difficult.

6.4 Next-Gen Geometry Pipeline

Although Compute Shader-based tessellation offers greater flexibility than the traditional hardware pipeline, it introduces notable overhead. Since it can't pass vertex data directly into the graphics pipeline, the output must be written to GPU memory, causing extra I/O and memory bandwidth pressure. In contrast, hardware tessellation writes vertex data directly to the GPU cache, avoiding this step and improving overall rendering efficiency.

Mesh Shader [33] somehow allow us to avoid unnecessary overhead while maintaining the flexibility and high parallel computational capability. As shown in the Figure 47, mesh shader, similar to hardware tessellation, can directly pass the output data to the subsequent rendering work, and due to the existence of task shader, it can dynamically dispatch the number of mesh shaders, thus fully utilizing the gpu's highly parallel computing capability. dispatch the number of mesh shaders, thus fully utilizing the gpu's highly parallel computing capability.

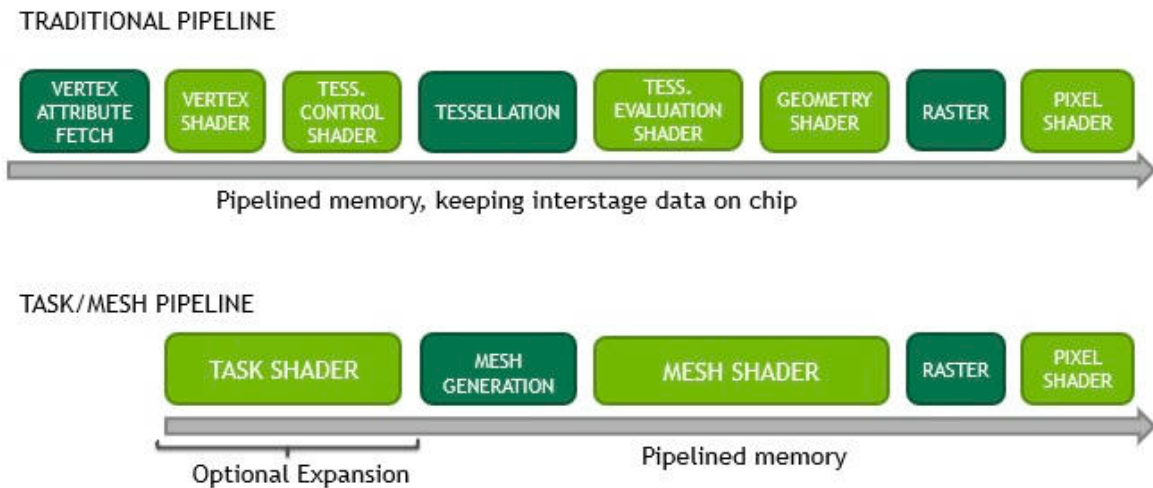


Figure 45: Traditional graphics pipeline vs. Mesh shader pipeline

In addition, the Mesh Shader Pipeline can effectively solve some of the performance bottlenecks that exist in traditional rendering pipelines. For example, in a traditional pipeline, the same vertex may be executed by multiple vertex shaders, resulting in completely unnecessary wasted computation [34]. At the same time, the fixed phases of the graphics pipeline lack sufficient flexibility and make it difficult to fully utilize the advantages of modern GPUs in massively parallel computation.

However, the output of mesh shaders are limited by hardware constraints—typically up to 256 vertices and 128 primitives per workgroup [33]. As a result, the size of reusable patterns is also constrained. To handle cases where a triangle’s tessellation level exceeds available patterns, we can iteratively split it until it fits within a supported pattern [29].

6.5 Visual effect

Even when using a 4K resolution displacement texture, at very high tessellation rates each triangle can become smaller than a single pixel. In such cases, the limited bit depth used to store scalar values may not provide enough precision, leading to artifacts as shown in Figure 48.

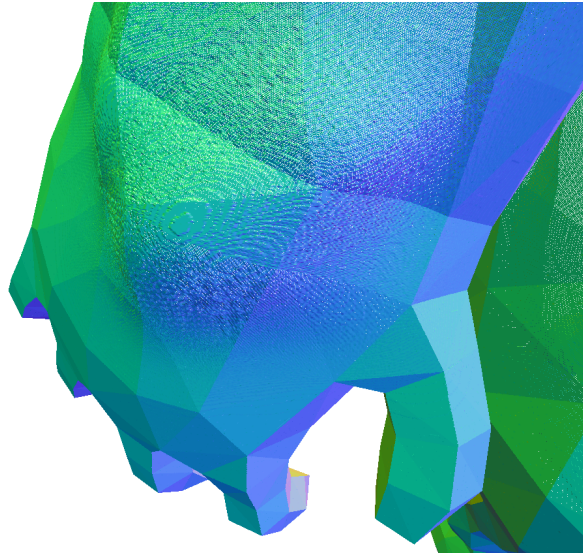


Figure 46: Displacement artifacts when tessellation is too high

As you can see from the previous results, since we only increased the density of the triangles and did not do any smoothing like the subdivision surface [35], visually, even though we interpolated and smoothed the vertex normal inside the triangles with the rasterizer, the silhouette of the model still looks sharp and angular, see Figure 49.

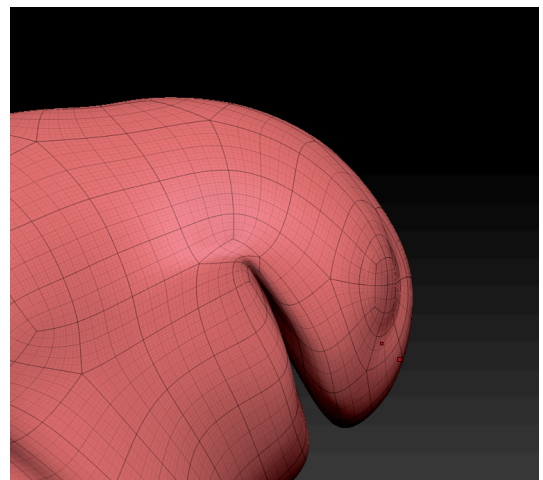
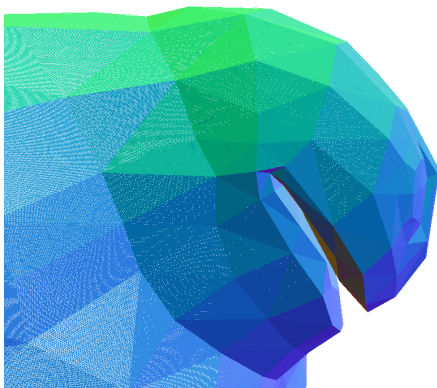


Figure 47: Side view from the same model of tessellation(left) and subdivision(right)

Common approaches to improve this are generally divided into two categories: one is directly rendering subdivision surfaces, and the other is constructing higher-order smooth surfaces based on flat triangles.

- [36] Displaced subdivision surfaces
- [37] Rendering subdivision surfaces using hardware tessellation
- [38] Time and Memory Efficient Displacement Map Extraction
- [39] quad tree
- [37] subd hw tess
- [40] subd in animation

- [41] doo subd
- [16] 3 subd
- [42] eval method
- [2] catmull subd
- [43] loop

Subdivision Surface is actually the iterative application of Subdivision Schemes [2, 16, 41, 43] on top of a coarse mesh to achieve surface smoothing by continuously generating and weighting vertex positions. These approaches are particularly computationally intensive in the case of adaptive subdivision. Especially for animated mesh [40], the Subdivision Level of each frame may be different due to the change of geometric complexity over time, which not only requires the dynamic generation of a large number of vertices, but also needs to offset the positions of existing vertices according to the weighting rule to ensure the surface smoothness. In this regard, studies such as Brainerd et al. [39] have proposed to pre-generate the Subdivision plan by using the adaptive quad tree structure, thus realizing efficient GPU rendering of Subdivision Surface in the GPU.

Another representative method is PN Triangle [7, 20, 44], which constructs higher-order surfaces based on planar triangles to achieve smoother visual effects with less computational cost. PN Triangle interpolates vertex positions and normals to construct quadratic Bezier surfaces, which significantly improves the angularity of planar triangles and is lighter in computation compared to the full Subdivision.

It is also lighter than a full Subdivision, making it suitable for real-time rendering. In addition to PN Triangle, recent years have seen a rise in shader-based surface subdivision and fitting techniques, such as the use of Bezier Patch, Gregory Patch [45], and other higher-order surface models.

Chapter 7

Conclusion

7.1 Key Findings

7.2 Research Questions

References

- [1] H. Hoppe, “Progressive meshes,” *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*. pp. 111–120, 2023.
- [2] E. Catmull and J. Clark, “Recursively generated B-spline surfaces on arbitrary topological meshes,” *Seminal graphics: pioneering efforts that shaped the field*. pp. 183–188, 1978.
- [3] C. Loop, “Smooth subdivision surfaces based on triangles,” 1987.
- [4] T. Boubekeur and C. Schlick, “Generic mesh refinement on GPU,” in *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS conference on Graphics hardware*, 2005, pp. 99–104.
- [5] R. Lenz, J. B. Cavalcante-Neto, and C. A. Vidal, “Optimized pattern-based adaptive mesh refinement using gpu,” in *2009 XXII Brazilian Symposium on Computer Graphics and Image Processing*, 2009, pp. 88–95.
- [6] T. Boubekeur and M. Alexa, “Phong tessellation,” *ACM SIGGRAPH Asia 2008 papers*. pp. 1–5, 2008.
- [7] A. Vlachos, J. Peters, C. Boyd, and J. L. Mitchell, “Curved PN triangles,” in *Proceedings of the 2001 symposium on Interactive 3D graphics*, 2001, pp. 159–166.
- [8] Microsoft Docs, “Direct3D 11 Features - Tessellation.” [Online]. Available at: <https://learn.microsoft.com/en-us/windows/win32/direct3d11/direct3d-11-features#tessellation>
- [9] Khronos Group, “OpenGL 4.3 Specification - Compute Shaders.” [Online]. Available at: https://www.khronos.org/opengl/wiki/Compute_Shader
- [10] J. Aaltonen, S. Laine, T. Karras, and T. Aila, “Reference Hierarchy for Ray Tracing.” [Online]. Available at: https://advances.realtimerendering.com/s2015/aaltonenhaar_siggraph2015_combined_final_footer_220dpi.pdf

- [11] B. Karis, R. Stubbe, and G. Wihlidal, “A Deep Dive into Nanite Virtualized Geometry.” [Online]. Available at: https://advances.realtimerendering.com/s2021/Karis_Nanite_SIGGRAPH_Advances_2021_final.pdf
- [12] T. Hák, S. Janoušková, and B. Moldan, “Sustainable Development Goals: A need for relevant indicators,” *Ecological indicators*, vol. 60, pp. 565–573, 2016.
- [13] M. Schwarz and M. Stamminger, “Fast GPU-based adaptive tessellation with CUDA,” in *Computer Graphics Forum*, 2009, pp. 365–374.
- [14] H. Bowles, D. Zimmermann, G. Noris, and B. Wang, “Crest: Novel Ocean Rendering Techniques in an Open Source Framework,” in *Siggraph 2017 Advances in Real-Time Rendering in Games*, 2017.
- [15] P. A. Studios, “Subdivision Surfaces.” [Online]. Available at: https://graphics.pixar.com/opensubdiv/docs/subdivision_surfaces.html
- [16] L. Kobbelt, “ $\sqrt{3}$ -subdivision,” in *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, 2000, pp. 103–112.
- [17] R. L. Cook, “Shade trees,” in *Proceedings of the 11th annual conference on Computer graphics and interactive techniques*, 1984, pp. 223–231.
- [18] Autodesk, “Vector Displacement Maps.” [Online]. Available at: https://download.autodesk.com/global/docs/softimage2014/en_us/userguide/index.html?url=files/GUID-21452F12-9509-4F96-BFC0-F63F8DDD6465.htm,topicNumber=d30e600255
- [19] T. Boubekur and C. Schlick, “A flexible kernel for adaptive mesh refinement on GPU,” in *Computer Graphics Forum*, 2008, pp. 102–113.
- [20] M. Schwarz, M. Stuginski, and M. Stamminger, “GPU-based rendering of PN triangle meshes with adaptive tessellation,” in *Proceedings of Vision, Modeling, and Visualization*, 2006, pp. 161–168.
- [21] J. Khoury, J. Dupuy, C. Riccio, and W. Engel, “Adaptive GPU tessellation with compute shaders,” *GPU Zen: Advanced Rendering Techniques*, vol. 2, pp. 3–17, 2019.
- [22] J. Dupuy, “Concurrent Binary Trees (with application to longest edge bisection),” *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 3, no. 2, pp. 1–20, 2020.
- [23] Microsoft Corporation, “Direct3D 11 Graphics - Win32 apps.” [Online]. Available at: <https://learn.microsoft.com/en-us/windows/win32/direct3d11/atoc-dx-graphics-direct3d-11>
- [24] A. Maggiordomo, H. Moreton, and M. Tarini, “Micro-mesh construction,” *ACM Transactions on Graphics (TOG)*, vol. 42, no. 4, pp. 1–18, 2023.

-
- [25] NVIDIA Corporation, “Micro-Mesh Basics.” [Online]. Available at: https://developer.download.nvidia.com/ProGraphics/nvpro-samples/slides/Micro-Mesh_Basics.pdf
- [26] C. Kubisch, “Micro-Mesh Rasterization.” [Online]. Available at: https://developer.download.nvidia.com/ProGraphics/nvpro-samples/slides/Micro-Mesh_Rasterization.pdf
- [27] NVIDIA Corporation, “Micro-Mesh Asset Pipeline.” [Online]. Available at: https://developer.download.nvidia.com/ProGraphics/nvpro-samples/slides/Micro-Mesh_Asset_Pipeline.pdf
- [28] NVIDIA Corporation, “NVIDIA Ada GPU Architecture.” [Online]. Available at: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>
- [29] NVIDIA, “NVIDIA® RTX Mega Geometry.” Accessed: February 6, 2025. [Online]. Available at: <https://github.com/NVIDIA-RTX/rtxmg.git>
- [30] J. Barczak, C. Benthin, and D. McAllister, “Dgf: A dense, hardware-friendly geometry format for lossily compressing meshlets with arbitrary topologies,” *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 7, no. 3, pp. 1–17, 2024.
- [31] F. Evans, S. Skiena, and A. Varshney, “Optimizing triangle strips for fast rendering,” in *Proceedings of Seventh Annual IEEE Visualization'96*, 1996, pp. 319–326.
- [32] I. Wald, “GPGPU-parallel re-indexing of triangle meshes with duplicate-vertex and unused-vertex removal,” *arXiv preprint arXiv:2109.09812*, 2021.
- [33] C. Kubisch, “Introduction to Turing Mesh Shaders.” [Online]. Available at: <https://developer.nvidia.com/blog/introduction-turing-mesh-shaders/>
- [34] M. Oberberger, B. Kuth, and Q. Meyer, “From Vertex Shader to Mesh Shader.” [Online]. Available at: https://gpuopen.com/learn/mesh_shaders/mesh_shaders-from_vertex_shader_to_mesh_shader/
- [35] B. Sharp, “Subdivision surface theory,” *Game Developer*, vol. 7, no. 1, pp. 34–42, 2000.
- [36] A. Lee, H. Moreton, and H. Hoppe, “Displaced subdivision surfaces,” in *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, 2000, pp. 85–94.
- [37] M. Nießner, *Rendering subdivision surfaces using hardware tessellation*. Verlag Dr. Hut, 2013.
- [38] V. Melapudi and P. Havaladar, “Time and Memory Efficient Displacement Map Extraction,” *ACM SIGGRAPH 2021 Talks*, pp. 1–2, 2021.
-

- [39] W. Brainerd, T. Foley, M. Kraemer, H. Moreton, and M. Nießner, “Efficient GPU rendering of subdivision surfaces using adaptive quadtrees,” *ACM Transactions on Graphics (TOG)*, vol. 35, no. 4, pp. 1–12, 2016.
- [40] T. DeRose, M. Kass, and T. Truong, “Subdivision surfaces in character animation,” *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*. pp. 801–810, 2023.
- [41] D. Doo, “A subdivision algorithm for smoothing down irregularly shaped polyhedrons,” *Computer Aided Design*, pp. 157–165, 1978.
- [42] J. Stam, “Evaluation of loop subdivision surfaces,” in *SIGGRAPH’98 CDROM Proceedings*, 1998.
- [43] M. Research, “Smooth Subdivision Surfaces Based on Triangles,” 2016. [Online]. Available at: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/thesis-10.pdf>
- [44] T. Boubekur, P. Reuter, and C. Schlick, “Scalar tagged PN triangles,” in *EUROGRAPHICS Short Papers*, 2005.
- [45] C. Loop, S. Schaefer, T. Ni, and I. Castano, “Approximating subdivision surfaces with Gregory patches for hardware tessellation,” *ACM SIGGRAPH Asia 2009 papers*. pp. 1–9, 2009.

Appendices

EXAMENSARBETE Real-Time Geometry Reconstruction with Compute Shader Tessellation

STUDENTER Jintao Yu

HANDLEDARE Kindahl Christian, Michael Doggett

EXAMINATOR Mattias Wallergård

POPULÄRVETENSKAPLIG SAMMANFATTNING **Jintao Yu**
